

Tempo: An ML-KEM to PAKE Compiler Resilient to Timing Attacks

Afonso Arriaga¹, Manuel Barbosa² and Stanislaw Jarecki³

¹ SnT - University of Luxembourg

afonso.delerue@uni.lu

² University of Porto (FCUP), INESC-TEC
and Max Planck Institute for Security and Privacy

mbb@fc.up.pt

³ University of California at Irvine

sjarecki@ics.uci.edu

Abstract. KEM-to-PAKE compilers have been recently proposed to leverage post-quantum KEM standardization efforts. These typically follow the Encrypted Key Exchange (EKE) paradigm, where the KEM public key is encrypted under a password. While KEM implementations generally aim to execute all secret-dependent computations in constant time, such guarantees do not always extend to computations that depend only on the public key, as public keys are generally assumed to be public. A notable example is ML-KEM, where public keys include a short seed ρ from which a large matrix is expanded prior to algebraic computations. This expansion procedure relies on rejection sampling which is, typically, implemented as a variable-time algorithm. However, compilers that follow the EKE paradigm must treat the public key as secret, since knowledge of the public key hidden in a ciphertext enables an offline dictionary attack on the password; therefore, instantiating these compilers with off-the-shelf implementations of ML-KEM or encrypting the public key under a password using variable-time methods (e.g., cycle walking or rejection sampling, as proposed in related PAKE literature) can be problematic.

In this paper we show two approaches which yield ML-KEM-based PAKEs resilient to timing attacks. First, we explore constant-time alternatives to ML-KEM rejection sampling, but as one might expect, such methods impose a performance penalty on ML-KEM. Our second approach introduces a new ML-KEM-to-PAKE compiler that mitigates this issue by design: our proposal transmits the seed ρ in the clear, decoupling password-dependent computations from the seed expansion step. This means that off-the-shelf implementations of ML-KEM can be used. Our new protocol **Tempo** builds on an idea from CHIC (Asiacrypt’24), which considered splitting the KEM public key, and extends the NoI protocol and proof (ePrint:2025/231) to show protocol simulation in the UC framework, assuming ML-KEM security under adversarially chosen seeds. We justify this assumption via a new hardness assumption, which we call Oracle-MLWE, and show that it is asymptotically equivalent to the MLWE problem in the Random Oracle Model.

Keywords: Password Authenticated Key Exchange (PAKE) · Timing Attacks · ML-KEM · Constant-Time · Rejection Sampling · Universal Composability · Post-Quantum

1 Introduction

Password Authenticated Key Exchange (PAKE) is a cryptographic primitive that enables two parties to establish a secure channel starting from a pre-shared (low-entropy) password,

and does not assume trust on any PKI. Originally introduced by the seminal work of Bellovin and Merritt [BM92] in 1992, PAKE protocols have gained popularity in more recent years as they found applications in electronic passports, Wi-Fi WPA3, password managers, browser synchronization, WhatsApp backups, Apple HomeKit, Keychain, and CarKey, Facebook Messenger chat history sharing, and in the context of IoT device enrollment.¹

KEM-to-PAKE compilers. Many practical PAKE protocol designs, e.g. PAK [BMP00], SPAKE2 [AP05], or CPACE [AHH21], are linked to the Diffie-Hellman protocol, which unfortunately is known not to withstand attacks with a large quantum computer. A transition to post-quantum cryptography has renewed the interest in PAKE design, and in particular in the KEM-to-PAKE compilers that leverage recent standardization efforts on post-quantum KEMs [MRR20, BCP⁺23, SGJ23, PZ23, AHHR24, ABJS24, JRX25, ABJ25].

Using the taxonomy on PAKE design classes from [HvO22], these KEM-to-PAKE compilers follow the *Encrypted Key Exchange* approach of Bellovin-Meritt [BM92] and, more specifically, the so-called *One-Encryption EKE* (OEKE) [BCP⁺23] where the password is used to encrypt only *one* key exchange flow, namely the KEM public key. (We will call the use of a symmetric primitive keyed only with the password as *password-encryption*.) Security of these protocols depends on KEM secrecy and anonymity, and whether the password-encryption domain can be aligned with the domain of KEM public keys (the intuition that pseudo-uniformity suffices is an over-simplification). These compilers either assume an Ideal Cipher (IC) as the password-encryption method [BPR00, BCP⁺23], a Half-Ideal Cipher (HIC), realized by a *modified* 2-round Feistel [SGJ23, ABJS24], or a Programmable-Once Public Function, realized by a 2-round Feistel [MRR20, JRX25, ABJ25]. The 2-round Feistel constructions operate as a randomized cipher, where both round functions are modeled as Random Oracles (RO), and the public key is fed to one wire of the Feistel network, while the other wire carries random coins. The *modified* 2-round Feistel replaces the XOR operation over the random coins with a small domain IC.²

There are two reasons for choosing 2-round Feistel rather than IC for password-encryption of KEM keys in an OEKE-style PAKE protocol.

First, instantiating an IC over the public key domain of post-quantum KEMs is non-trivial. On the one hand, the public key space of post-quantum KEMs is much larger than that of a standard block cipher. One could implement an IC on a large bytestring domain using IC domain extenders [CDMS10], but the recursive construction of [CDMS10] would add a significant cost for such large domain [AMO⁺26]. One could also implement this IC using an 8-round Feistel network (if the Feistel round functions are modeled as random oracles then an 8-round Feistel is indistinguishable from an Ideal Cipher [DS16]), but an 8-round Feistel is strictly less efficient than a 2-round Feistel which is shown to suffice in [JRX25, ABJ25]. Moreover, if the IC is instantiated using a cipher whose input space does not exactly match the public key space, an adversary can decrypt the password-authenticated ciphertext under candidate passwords and discard those that do not yield a plaintext within the public key space. (This attack does not involve timing!) In fact, the difficulty of instantiating an IC over arbitrary finite domains was seen as the reason that prevented the wide use of the (O)EKE-style PAKE protocols (see e.g. [HvO22, Section 3.1]). The 2-round Feistel constructions minimize the domain compatibility problems, since the password-encryption operation can be done efficiently over arbitrarily large inputs *and* it can be generalized to an arbitrary group rather than operating only over bitstrings.

Secondly, whereas we have many techniques to deal with a quantum analogue of a random oracle (RO) [Zha19, CFHL21], the same does yet not hold for the ideal cipher

¹We invite the reader looking for pointers to these PAKE deployments to visit <https://github.com/fancy-cryptography/fancy-cryptography>.

²Technically, CHIC [ABJS24] de-randomizes the cipher by splitting the KEM public key and using one of the public key components in place of the random coins.

(IC) [Unr23], hence a classical security proof in the RO model is more likely to be ‘lifted’ to the security proof against quantum adversaries than the IC model proof.

What was public is now secret: timing leakage on public keys. Counterintuitively, although the security proofs of the RO model KEM-to-PAKE constructions [JRX25, ABJ25] and the IC model constructions [BCP⁺23, SGJ23, ABJS24] can in theory be instantiated with ML-KEM—the first post-quantum KEM standard established by NIST [SAB⁺22]—a practical deployment that relies on an off-the-shelf ML-KEM implementation will most likely be insecure in practice due to timing attacks! We explain why this is the case next.

ML-KEM is highly efficient and has comparatively short public keys, making it the primary candidate for KEM-to-PAKE compilers, which result in protocols with low computational and bandwidth requirements. ML-KEM public keys are “compressed” and consist of a group element $t \in (\mathcal{R}_q^{256})^k$ where $q = 3329$, a prime, and $k \in \{2, 3, 4\}$, and a random 256-bit seed ρ , which is expanded into a matrix $A \in (\mathcal{R}_q^{256})^{k \times k}$. The seed-to-matrix expansion is performed during key generation, encapsulation and decapsulation. Per the FIPS 203 specification, this expansion is done via algorithm `SampleNTT` [SAB⁺22, Algorithm 7], which describes a variable-time rejection sampling procedure. Indeed, timing leaks in matrix expansion are not considered problematic in FIPS 203, because this algorithm only processes public information; moreover, to the best of our knowledge most, if not all, popular implementations of ML-KEM should be assumed to leak the seed ρ via timing variations in the matrix expansion step. This immediately implies that any password-encryption of an ML-KEM public key that is processed in this way will be exposed to an offline dictionary attack on the password!

In more detail, consider the following timing attack on an OEKE-style PAKE and, for simplicity of exposition, assume that an attacker can perfectly time the PAKE initiator party and extract timing information correlated with the public-key. Let us call the protocol initiator Alice and the responder Bob. Alice first generates a KEM public key pair, and password-encrypts the public key—denoted `apk` for *authenticated public key*—before transmitting it to Bob. The adversary can decrypt `apk` under every candidate password to obtain a candidate public key, and then check which of those keys is consistent with the timing leakage obtained from Alice to recover the password. Similarly, on the responder side, Bob will perform password-decryption and use the recovered public key to perform an encapsulation, which will permit establishing a shared key with Alice. However, if the attacker times Bob and obtains leakage about the public key used in encapsulation, again it can try all passwords on the `apk` received by Bob until a consistent public key is found.

As a conclusion: off-the-shelf implementations of ML-KEM do not consider public keys as secrets, and hence they do not include protections against timing attacks targeting the seed-to-matrix expansion. This means that specialized implementations of ML-KEM will be required to securely instantiate the existing OEKE-style PAKE constructions.

Timing leakage from password-encryption. So far we considered timing leakage in the implementation of the KEM subcomponent in OEKE-style PAKEs. However, the implementation of the password-encryption step itself is problematic. Indeed, as seen above, if the distribution of KEM public keys is not computationally indistinguishable from the uniform distribution over bit strings—which unfortunately is not true for ML-KEM³—special care must be taken when instantiating the ideal objects assumed by PAKE constructions to avoid trivial attacks. We will now see that the solutions often raise new problems with respect to timing attacks as well.

For example, one can implement a cipher over an arbitrary finite domain using ‘cycle-walking’ [BR02], which involves iterating the cipher until reaching a point in the domain. This is precisely how the authors of CAKE/OCAKE [BCP⁺23] proposed to in-

³Note that, unlike FrodoKEM [LBES25], which employs a power-of-two integer modulus and is therefore naturally representable as a bitstring, ML-KEM uses a prime modulus q , which makes implementing a block cipher over its domain non-trivial.

stantiate the IC, but this makes encryption variable-time. Using Feistel for password-encryption [SGJ23, ABJS24, JRX25, ABJ25] dispenses with a block cipher over the KEM public key domain, but it requires an RO hash onto the same domain. The authors of [ABJS24] proposed instantiating this hash function in their CHIC[ML-KEM] protocol by adapting the seed-to-matrix expansion procedure from the ML-KEM reference code mentioned above.⁴ Consequently, for the same reason as previously discussed for ML-KEM, their implementation of the hashing procedure is vulnerable to timing attacks and, again, specialized implementations of the rejection sampling process need to be considered.

An alternative solution to implementing password-encryption is to preprocess ML-KEM public keys with an encoding that guarantees that they can be naturally represented as bit strings. Recent works [GSV24a, GSV24b] propose such encoding algorithms for ML-KEM public keys s.t. an encoding of a random public key is indistinguishable from a random bytestring.⁵ Among these, the KemeleonNR encoding variant [GSV24b] avoids rejection sampling, but it relies on multi-precision division, which raises different challenges when it comes to constant-time implementation.⁶

Recent developments and further motivation. A recent IETF Internet-Draft [VJW25] proposes a hybrid PAKE protocol [LL24, HR24] that combines CPace [HL19] with the PAKE protocol NoIC [ABJ25], an OEKE-style KEM-to-PAKE compiler which uses 2-round Feistel for password-encryption of KEM’s public keys. IETF draft [VJW25] instantiates NoIC with ML-KEM and uses KemeleonNR to encode public keys as byte strings. In light of the above discussion, this proposal raises several interesting questions which we address in this paper. The first question concerns the choice of ML-KEM and the need of specialized implementations that remove timing leaks during public-key expansion: in which ways can this be done, and what is the inherent performance cost?⁷ The second question is whether there are alternative OEKE-style PAKE designs that remove the need for such specialized implementations. We propose a new design that has this property and discuss under what conditions it can be instantiated with vanilla implementations of ML-KEM.

Our contributions. This paper contains two main contributions:

(I) First, we specify three constant-time algorithms for public-key expansion in ML-KEM key generation, i.e. for seed-to-matrix expansion that maps a 256-bit random seed ρ into matrix $A \in (\mathcal{R}_q^{256})^{k \times k}$. The first algorithm is a (constant-time) functional equivalent to the variable-time public-key expansion algorithm in the ML-KEM FIPS 203 specification, i.e. algorithm SampleNTT [SAB⁺22, Algorithm 7]. The other two are based on modular arithmetic and functionally diverge from the FIPS 203 standard (i.e. they do not generate the same matrix from the same seed ρ), but one of them offers faster constant-time execution. We benchmark all three against the baseline SampleNTT from FIPS 203, which uses variable-time rejection sampling, and measure their impact on ML-KEM and NoIC[ML-KEM].

(II) Secondly, we introduce protocol Tempo, a variant of NoIC that avoids timing attacks by explicitly revealing the seed ρ , thus making the expansion process to matrix A public, as originally intended in ML-KEM. We identify the new requirements that the underlying KEM must satisfy, we show that ML-KEM meets them under a variant of MLWE hardness assumption we call *Oracle-MLWE*, and we show that if these requirements hold then

⁴<https://github.com/pq-crystals/kyber>

⁵Kemeleon encoding is also applicable to ML-KEM ciphertexts, although in our context this is only relevant for EKE-style PAKEs that password-encrypt both the KEM public key and the KEM ciphertext, such as in CAKE [BCP⁺23] and EKE-PRF [JRX25] protocols.

⁶We note that these encodings should be seen as a tradeoff: they give rise to larger public keys and incur on an extra computational cost, but they facilitate the implementation of the password-encryption step because one can implement the 2-round Feistel [MRR20, JRX25, ABJ25] (or the modified 2-round Feistel [SGJ23, ABJS24]) over bit strings.

⁷This discussion also raises the question of whether an instantiation based on FrodoKEM might be preferable overall.

Tempo realizes the standard Universally Composable (UC) PAKE functionality of Canetti et al. [CHK⁺05]. We implement and benchmark **Tempo** against timing-attack-protected NoC on an ARM Cortex-M4 platform.

Organization. Section 2 includes the three constant-time SampleNTT algorithms and their performance benchmarks. Section 3 presents protocol **Tempo**, including the KEM requirements it relies on, and its performance benchmarks. Section 4 provides concluding remarks. We defer to Appendix (A) the baseline SampleNTT algorithm from FIPS 203, (B) the UC PAKE functionality of [CHK⁺05], and (C) a comparison of game hops in the security proof of **Tempo** and NoC [ABJ25].

2 Implementing SampleNTT in constant-time

We established that all previous KEM-to-PAKE compilers [BCP⁺23, SGJ23, ABJS24, JRX25, ABJ25] instantiated with ML-KEM [SAB⁺22] are potentially vulnerable to offline dictionary attacks via timing either Alice or Bob. This vulnerability can be traced back to the SampleNTT algorithm [SAB⁺22, Algorithm 7] in ML-KEM, which employs a rejection sampling procedure that is usually not implemented in constant-time, and it can be compounded by the password-encryption method used to encrypt the KEM public key.

However, as explained above, SampleNTT can also be used to hash onto the public key space of ML-KEM and thus implement the 2-round Feistel password-encryption method of the recent KEM-to-PAKE compilers [JRX25, ABJ25]. Therefore, a constant-time implementation of SampleNTT will effectively permit removing timing leaks from both sources (1) and (2) above.⁸

In this section, we discuss a SampleNTT implementation that is constant-time and functionally equivalent to the one in FIPS 203 [SAB⁺22], as well as two alternative constant-time algorithms, one of them derived from the KameleonNR encoding, that produce polynomials with coefficients that are statistically close to uniform in $\mathbb{Z}_q$. We note that these latter two alternatives break compatibility with the ML-KEM specification, but result in variants of ML-KEM that meet the same level of theoretical security.

The reference algorithm is SampleNTT, as specified in [SAB⁺22, Algorithm 7] and provided in Appendix A for ease of reference and comparison. It serves as baseline against which we compare our three proposed alternatives. Each algorithm is used to generate k^2 entries of matrix $A \in (\mathbb{Z}_q^{256})^{k \times k}$, but each algorithm samples one entry at a time, hence in the explanations below, and in the algorithm specifications, we treat each algorithm as generating a single vector $a \in \mathbb{Z}_q^{256}$.

The three algorithms are as follows:

- **Algorithm FLS:** The SampleNTT algorithm from the ML-KEM specification [SAB⁺22] expands a 34-byte seed B into 256 integers modulo q using rejection sampling over the output of an eXtendable Output Function (XOF). The first 32 bytes of the seed are uniformly random, and the remaining 2 bytes are reserved to encode the row and column indices of the matrix A . The XOF function is squeezed 3 bytes at a time. These 3 bytes are split into two 12-bit candidates, which are accepted as valid polynomial coefficients if they are in the range $[0, 3329)$, and rejected otherwise. The process is repeated until all coefficients of the polynomial have been sampled. FIPS 203 standard allows an optional bound on the while-loop to be set, halting the process after 280 iterations [SAB⁺22, Appendix B], which introduces an estimated failure probability of

⁸In particular, a constant-time implementation of SampleNTT enables constant-time hashing onto the group over which the KEM public key is defined, which means that password-encryption can be performed securely without resorting to KameleonNR [GSV24b] as proposed in the IETF draft mentioned above [VJW25]. This saves both computational cost and bandwidth.

Algorithm 1 (FLS: Fixed Loop Sampler)

This algorithm refactors the variable-time rejection sampling `SampleNTT` [SAB⁺22, Algorithm 7], rewriting it in a constant-time style with fixed loops.

Input: byte array $B \in \mathbb{B}^{34}$

Output: array $a \in \mathbb{Z}_q^{256}$

```

1: ctx ← XOF.Init()
2: ctx ← XOF.Absorb(ctx, B)
3: counter ← 0
4: for iter = 0 to 279 do
5:   (ctx, C) ← XOF.Squeeze(ctx, 3)
6:   d1 ← ((C[0] ≫ 0) | (C[1] ≪ 8)) & 0xFFF
7:   d2 ← ((C[1] ≫ 4) | (C[2] ≪ 4)) & 0xFFF      ▷ taken from Kyber reference code
8:   acceptabled1 ← int(d1 < q)
9:   acceptabled2 ← int(d2 < q)
10:  for (d, acceptable) ∈ {(d1, acceptabled1), (d2, acceptabled2)} do
11:    flag ← 0
12:    for j = 0 to 255 do
13:      match ← int(j == counter)
14:      mask ← match * acceptable
15:      a[j] ← a[j] * (1 - mask) + d * mask
16:      flag ← flag + mask
17:    end for
18:    counter ← counter + flag
19:  end for
20: end for
21: return a

```

2^{-161} , corresponding to the event that not all polynomial coefficients are sampled. (We include the `SampleNTT` algorithm in Appendix A for reference.)

The first algorithm we describe as *Fixed Loop Sampler* (FLS) rearranges the control flow of `SampleNTT` to avoid data-dependent branching and conditional storage by removing the two separate conditions that terminate the while-loop (i.e., when the output array is filled or the loop bound is reached). Instead of stopping once enough samples are collected, it always performs 280 sampling iterations. As in the original pseudocode, each iteration expands via the XOF 3 bytes at a time, which are split into two 12-bit values. To ensure constant-time execution, the algorithm loops over all positions in the output array for each candidate value, assigning it to the next unfilled position. Pseudocode is crafted carefully to assign an acceptable candidate at most once. In short, the algorithm: (i) avoids branching when selecting a candidate; (ii) avoids data-dependent addressing when selecting the storage position into the output array; and (iii) avoids modular arithmetic operations (e.g., division) that may not be constant-time unless carefully implemented. This algorithm is functionally equivalent to FIPS 203 `SampleNTT`, meaning that on input the same seed, both algorithms output exactly the same coefficients.

- **Algorithm RMD:** We refer to the second method as *Repeated Modular Division* (RMD). This approach deviates from the ML-KEM specification and adopts a strategy similar to the `VectorDecodeNR` method described in the non-rejection sampling variant of Kameleon encodings [GSV24b], which in turn is adapted from [Tib14]. The seed B is first expanded into a large integer of 400 bytes (= 3200 bits). Coefficients are then ex-

Algorithm 2 (RMD: Repeated Modular Division)

In this approach, the seed B is expanded into a very large integer. Then coefficients are extracted by repeatedly applying modular reduction and integer division, where at each iteration the remainder of the previous quotient yields a new coefficient. Here, $\text{Divmod}(z, \text{KYBER_Q})$ returns (q, r) with $z = q \cdot \text{KYBER_Q} + r$ and $0 \leq r < \text{KYBER_Q}$.

Input: byte array $B \in \mathbb{B}^{34}$

Output: array $a \in \mathbb{Z}_q^{256}$

```

1:  $\text{ctx} \leftarrow \text{XOF.Init}()$ 
2:  $\text{ctx} \leftarrow \text{XOF.Absorb}(\text{ctx}, B)$ 
3:  $(\_, C) \leftarrow \text{XOF.Squeeze}(\text{ctx}, 400)$   $\triangleright$  400 is the total number of bytes needed to encode
   256 coefficients + 204 bits for security
4:  $z \leftarrow \text{int}(C)$ 
5: for  $j = 0$  to 255 do
6:    $(q, r) \leftarrow \text{Divmod}(z, \text{KYBER\_Q})$ 
7:    $a[j] = r$ 
8:    $z \leftarrow q$ 
9: end for
10: return  $a$ 

```

tracted by repeatedly applying modular reduction and integer division: at each iteration, the remainder of the previous quotient yields a new coefficient. The initial 400-byte integer is chosen to ensure it is sufficient to encode all 256 coefficients of the polynomial, with some extra bits for security. Note that $\lceil \log_2(\text{KYBER_Q}^{\text{KYBER_N}}) \rceil = 2996$ bits are necessary to extract all coefficient using this method, where $\text{KYBER_Q} = 3329$ is ML-KEM prime and $\text{KYBER_N} = 256$ is the degree of ML-KEM polynomials. This implies that the additional 204 bits (for a total of 3200 bits or 400 bytes) provide a statistical security margin. Specifically, this variant yields encoded values whose statistical distance from uniform is at most $2^{-(\lambda=204)}$. To adjust the security parameter λ , it suffices to expand B to a larger or smaller integer.

- **Algorithm SMD:** We refer to the third method as *Single Modular Devision* (SMD). Previous method, Algorithm RMD, requires modular arithmetic over 400-byte integers, which may be unsuitable for application on constrained devices, e.g. electronic passports. Our last method avoids such large integers by squeezing the XOF 24 bytes at the time, enough to extract a single coefficient with a security parameter of $\lambda = (24 * 8) - \lceil \log_2(\text{KYBER_Q}) \rceil = 192 - 12 = 180$. Again, this method deviates from the ML-KEM specification.

Implications for security. Algorithms RMD and SMD sample matrix entries a in $\mathcal{R}_q^{256}$ from a distribution whose statistical distance from uniform is upper-bounded respectively by $\delta = 2^{-204}$ and $\delta = 2^{-180}$. If the ML-KEM parameters are chosen to guarantee security bound ϵ against some KEM security notion, e.g. correctness, indistinguishability, or anonymity, then the security bound of adapted ML-KEM when the public-key is generated with statistical skew δ grows to at most $\epsilon + \delta$. Hence, adopting the alternative public-key expansion methods above has a negligible impact on ML-KEM security.

Experimental Results. As a first validation step, we implement and benchmark the above algorithms on a laptop with a 1.8 GHz Intel Core i7 running Debian 13 (minimal installation), with the benchmark script restricted to isolated cores to minimize cycle variance, using OpenSSL as a representative of a typical cryptographic multi-

Algorithm 3 (SMD: Single Modular Division)

In this method, the seed B is expanded into a 192-bit string. Since each coefficient requires 12 bits, the extra 180 bits ensure that the extracted values have a statistical distance of at most 2^{-180} from the uniform distribution. Coefficients are extracted using modular reduction. Here, $\text{Mod}(d, \text{KYBER_Q})$ returns $r = d - \text{KYBER_Q} \times \lfloor d/\text{KYBER_Q} \rfloor$.

Input: byte array $B \in \mathbb{B}^{34}$

Output: array $a \in \mathbb{Z}_q^{256}$

```

1: ctx ← XOF.Init()
2: ctx ← XOF.Absorb(ctx, B)
3: j ← 0
4: while j < 256 do
5:   (ctx, C) ← XOF.Squeeze(ctx, 24)
6:   d ← int(C)
7:   r ← Mod(d, KYBER_Q)
8:   a[j] ← r
9:   j ← j + 1
10: end while
11: return a

```

precision arithmetic implementation. Indeed, OpenSSL is one of the most widely deployed cryptographic libraries. Its code is well optimized and written in portable C, but it is also large and may not be suitable for constrained environments such as those of some application-specific uses of PAKE, such as electronic passports.⁹ Later in the paper, we provide more realistic benchmarks of the relevant PAKE protocols on Cortex M4 Nucleo-L4R5ZI board with 2MB of flash memory—which is less than the size of the OpenSSL library itself—and we will therefore not rely on OpenSSL. The source code used to produce these benchmarks is open-sourced (public repository available at <https://github.com/afonsoarriaga/ProjectTempo>) and relies on the reference implementation of ML-KEM available from <https://github.com/pq-crystals/kyber>.

Our baseline is the variable-time **SampleNTT** algorithm from the ML-KEM reference implementation. We compare our constant-time implementations of Algorithm **FLS** (FIPS 203 compatible and functionally equivalent to **SampleNTT**), Algorithm **RMD** (Repeated Modular Division method, implemented using OpenSSL), and Algorithm **SMD** (Single Modular Division method, implemented without dependencies using Barrett reduction [MvOV96, Algorithm 14.42]) with precomputations for $\text{KYBER_Q} = 3329$.

Benchmarks compare sampling methods across parameter sets with respect to: (i) matrix generation from seed ρ to A ; (ii) the impact on ML-KEM (keygen+encaps+decaps) reference code when switching the matrix generation method; and (iii) the impact on **NoIC** protocol (start+resp+end) when instantiated with ML-KEM and each sampling method (here, the selected sampling method is also used to instantiate the **NoIC** hash function H_1 that maps into the public-key space [ABJ25, see the **NoIC** protocol]). Results are presented in Table 1. More aggressive optimizations are certainly possible, but they must be applied carefully to avoid introducing time variations to the code. These results should therefore be viewed as a preliminary analysis of the trade-offs, but the overall conclusions are unlikely to change significantly.

Unsurprisingly, variable-time rejection sampling is the fastest method to expand the seed ρ , by around 7 to 8-fold compared to the second most efficient method, which is Algorithm **SMD**. Since ML-KEM key generation, encapsulation, and decapsulation all

⁹For an example of passport chips intended for the post-quantum era, see <https://web.archive.org/web/20250701110847/https://javacardforum.com/wp-content/uploads/2024/11/jcf-2024-electronic-passport.pdf>.

rely on **SampleNTT** (with decapsulation requiring re-encrypting, and therefore public key expansion, as part of the Fujisaki-Okamoto transform), this affects all KEM algorithms, so replacing the sampling method has a significant overall impact on ML-KEM. It is also clear from these preliminary results that the method based on Algorithm **RMD** is the least efficient option, performing worse than Algorithm **FLS**, which adheres to the specifications in the standard, so we will not consider it further: note also that Algorithm **RMD** is the only method that truly requires dealing with very large numbers and full division, so avoiding it also allows us to drop the dependency on OpenSSL or other similarly generic multi-precision arithmetic library.

Table 1: Benchmarks (in cycles on a laptop with a 1.8 GHz Intel Core i7 running Debian 13, averaged over 1000 runs $\pm$ relative standard deviations) comparing sampling methods across parameter sets: (i) matrix generation from seed ρ to A ; (ii) impact on ML-KEM (keygen+encaps+decaps) reference code when replacing `gen_a`; and (iii) impact on NoIC (start+resp+end) instantiated with ML-KEM and the new sampling method. Sampling methods: **FIPS 203** (variable-time rejection sampling [NIS23, Algorithm 7]), **FLS** (fixed loop sampler, constant-time refactor of [NIS23, Algorithm 7]), **RMD** (recursive modular division over a 3200-bit integer, implemented with OpenSSL) **SMD** (single modular division over 192-bit integers using the Barrett reduction method [MvOV96, Algorithm 14.42]).

	Implementations of SampleNTT			
	Algorithm FIPS 203	Algorithm FLS	Algorithm RMD	Algorithm SMD
	standalone, variable-time	standalone	OpenSSL	standalone
Avg time to generate matrix A $\forall i, j \in [0, k), A[i, j] \leftarrow \text{SampleNTT}(\rho[i j])$, where $k \in \{2, 3, 4\}$ is determined by the ML-KEM security parameter λ				
MLKEM-512.gen_a	68 974 $\pm$ 2.12%	487 938 $\pm$ 0.13%	1 174 719 $\pm$ 0.93%	484 339 $\pm$ 0.04%
MLKEM-768.gen_a	136 485 $\pm$ 1.09%	1 080 660 $\pm$ 0.09%	2 621 815 $\pm$ 0.50%	1 072 411 $\pm$ 0.23%
MLKEM-1024.gen_a	232 905 $\pm$ 0.82%	1 906 313 $\pm$ 0.15%	4 649 635 $\pm$ 0.34%	1 895 492 $\pm$ 0.05%
Avg time to execute MLKEM (keygen+enc+dec) SampleNTT used to (re-)generate matrix A				
MLKEM-512	712 717 $\pm$ 0.38%	1 972 312 $\pm$ 0.05%	4 030 149 $\pm$ 0.30%	1 959 798 $\pm$ 0.05%
MLKEM-768	1 213 602 $\pm$ 0.41%	4 052 170 $\pm$ 0.08%	8 622 886 $\pm$ 0.21%	4 014 919 $\pm$ 0.07%
MLKEM-1024	1 803 419 $\pm$ 0.33%	6 835 985 $\pm$ 0.06%	15 064 128 $\pm$ 0.53%	6 796 320 $\pm$ 0.06%
Avg time to execute NoIC PAKE (start+resp+end) SampleNTT used both to (re-)generate matrix A and instantiate H_1				
NoIC[MLKEM-512]	973 654 $\pm$ 0.29%	2 650 184 $\pm$ 0.12%	5 392 876 $\pm$ 0.30%	2 634 712 $\pm$ 0.13%
NoIC[MLKEM-768]	1 575 863 $\pm$ 0.35%	5 033 625 $\pm$ 0.06%	10 647 686 $\pm$ 0.14%	5 011 368 $\pm$ 0.06%
NoIC[MLKEM-1024]	2 295 495 $\pm$ 0.29%	8 162 881 $\pm$ 0.06%	17 752 864 $\pm$ 0.17%	8 117 978 $\pm$ 0.06%

3 Tempo PAKE

As discussed earlier, an active IETF Internet-Draft [VJW25] proposes a hybrid PAKE protocol that combines CPace [HL19] with NoIC [ABJ25], instantiated using ML-KEM-768 with the public keys encoded using the Kameleon binary uniform encoding [GSV24b]. (The draft refers to ML-KEM-768 with Kameleon key encoding as ‘BUKEM-768’, and to this instantiation of NoIC as ‘OQUAKE’.) The draft chose NoIC for its simplicity and close

to optimal computational and bandwidth efficiency.¹⁰ ML-KEM is the ideal candidate for instantiating NoIC because it is a current standard, it is widely available in cryptographic software libraries, possesses all the necessary properties, is computationally efficient, and has relatively small public keys and ciphertexts. However, one concern remains: It employs a rejection sampling procedure to expand the ρ component of the public key, which is usually not protected against timing attacks. Indeed, its reference code and all implementations that we are aware of—with the exception of our own implementation that follows the blueprint presented in Algorithm FLS—leak information about the ρ component in the public key through timing information, which in turn exposes the NoIC protocol to a timing attack. This raises the following question:

Can we build an ML-KEM-to-PAKE compiler that mitigates password-dependent runtime variations by transmitting the seed ρ in clear, and if so, at what cost in terms of the requirements on the KEM?

Design choices. We answer the question by designing a new PAKE protocol we call **Tempo**. Building on a long line of works that compile KEMs into PAKE protocols [BCP⁺23, SGJ23, ABJS24, JRX25, ABJ25], our **Tempo** protocol also builds on KEM. In particular, it closely follows the approach of NoIC [ABJ25] (and others) in several aspects:

(1) *OEKE paradigm* [BCP03]: Both NoIC and **Tempo** follow the OEKE design, where Alice generates a key pair, encrypts the public key under the shared password, and sends the resulting ciphertext to Bob. Bob decrypts it to recover Alice’s public key, uses it to encapsulate a shared secret, and returns a ciphertext along with an authentication tag. Alice decapsulates the ciphertext using her secret key, verifies the tag, and both parties derive a session key from the resulting shared secret and the communication transcript.¹¹ The original EKE paradigm [BM92] password-encrypts Bob’s ciphertext instead of relying on an authentication tag to bind Bob’s response to a password. However, since in post-quantum KEMs the public key space does not coincide with the ciphertext space, this approach requires an instantiation of password-encryption over KEM ciphertexts, which OEKE avoids.¹²

(2) *Password encryption with Feistel* [MRR20]: First proposed by the work of McQuoid, Rosulek and Roy [MRR20], both NoIC [ABJ25] and **Tempo** protocols use a 2-round Feistel network to password-encrypt Alice’s public key, avoiding the use of an Ideal Cipher. As we explain in Section 1, it is challenging to instantiate IC over the public key domains of post-quantum KEMs. Furthermore, it is unclear how to deal with quantum attackers in the Ideal Cipher model [Unr23], and there appear to be no known techniques for lifting PAKE security proofs that rely on IC to the quantum setting [HHKR25].

(3) *Public key is split and only one part is password-encrypted*: Protocol CHIC [ABJS24] introduced the idea of splitting the public key $\text{pk} = (\rho, t)$ and feeding ρ and t into two separate wires of a *modified* 2-round Feistel (m2F), effectively de-randomizing password-encryption and saving bandwidth. Protocol NICE-PAKE [AAA⁺24] also splits the KEM public key but it password-encrypts only the seed ρ part of the key. Since ρ is a short bit-string, its password-encryption is easy and fast, but the protocol is still subject to

¹⁰The computational overhead over the KEM is a 2-round Feistel to password-encrypt/decrypt the public key, and the bandwidth overhead is the Feistel randomness and an authentication tag, totaling 80 Bytes for security parameter $\lambda = 128$.

¹¹There are some variations in the literature regarding the elements that are used for deriving the authentication tag and session key. NoIC [ABJ25] opts for strong binding by including full session identifiers, the communication transcript, the password, and the shared secret. OCAKE [BCP⁺23] and CHIC [ABJS24] exclude the password from these inputs, which complicates the security proof but it lets Alice erase the password from memory after sending her initial message.

¹²OEKE adds a small bandwidth overhead to transmit the authentication tag, but password-encryption over the ciphertext space, in addition to being onerous and providing another avenue for timing attacks, would also add bandwidth overhead, e.g. for randomness in encryption via a Feistel.

a timing attack leading to an offline dictionary attack on the password.¹³ Similarly to CHIC [ABJS24] and NICE-PAKE [AAA⁺24], **Tempo** splits the public key into ρ and t , but it operates on this split key differently, with the goal of avoiding time-variable execution that depends on the secret password. Specifically, **Tempo** password-encrypts only the $\mathcal{R}_q^{256}$ vector t , while revealing seed ρ in the clear. This way **Tempo** avoids time-variable operations that depend on the secret password. Consequently, it doesn't matter if the adversary measures the timing of either party performing the variable-time seed expansion, because the seed is now public and processed independently of the secret password.

3.1 KEM with splittable keys

Protocol **Tempo**, shown in Fig. 2, crucially relies on KEM accompanied by procedure **Split** which satisfies the requirements we explain in this section.

First, recall that a key encapsulation mechanism (KEM) is a triple of algorithms $\text{KEM} = (\text{Keygen}, \text{Encap}, \text{Decap})$ s.t. for security parameter λ algorithm $\text{Keygen}(1^\lambda)$ generates a key pair (sk, pk) , algorithm $\text{Encap}(\text{pk})$ given the public key generates a ciphertext/key pair (c, K) , and $\text{Decap}(\text{sk}, c)$ outputs K . The correctness requirement is that if $(\text{sk}, \text{pk}) \leftarrow \text{Keygen}(1^\lambda)$, $(c, K) \leftarrow \text{Encap}(\text{pk})$, and $K' \leftarrow \text{Decap}(\text{sk}, c)$ then $K' = K$. However, as all LWE-based constructions, ML-KEM requires a relaxation of $(1 - \delta)$ -correctness, for δ negligible in λ , which says that the probability that $K = K'$ in the above experiment is at least $(1 - \delta)$, where the probability goes over the randomness of all algorithms.

Protocol **Tempo** relies on a KEM that supports additional efficient algorithms **Split** and **Keygen2** with the following syntax. Let $\mathcal{PK}$ denote the domain of the public keys generated by $\text{Keygen}(1^\lambda)$. (Set $\mathcal{PK}$ is implicitly indexed by λ but we omit it from the notation.) Let $\mathcal{PK}_t$ and $\mathcal{PK}_\rho$ be two sets, also implicitly indexed by λ , s.t. **Split** is a (possibly randomized) map $\text{Split} : \mathcal{PK} \rightarrow \mathcal{PK}_t \times \mathcal{PK}_\rho$ with an efficiently computable deterministic inverse $\text{Split}^{-1} : \mathcal{PK}_t \times \mathcal{PK}_\rho \rightarrow \mathcal{PK}$ s.t. for all $pk \in \mathcal{PK}$, $\text{Split}^{-1}(\text{Split}(pk)) = pk$. Algorithm **Keygen2** takes as input a security parameter and an element ρ and generates a KEM key pair (sk, pk) .

Algorithms **Split**, **Keygen2** must satisfy two further constraints: First, if pk is generated by $\text{Keygen2}(1^\lambda, \rho)$ then pk must contain the same value ρ after it is split via algorithm **Split**. Second, key pairs generated by **Keygen2** given a random ρ must be distributed identically to those generated via **Keygen**:¹⁴

Definition 1 (Splittable KEM). We call a key encapsulation mechanism KEM *splittable* if in addition to algorithms **Keygen**, **Encap**, **Decap** it supplies algorithms **Split** and **Keygen2** with accompanying domains $\mathcal{PK}_t, \mathcal{PK}_\rho$ s.t. (1) For any λ and $\rho \in \mathcal{PK}_\rho$, if $(\text{sk}, \text{pk}) \leftarrow \text{Keygen2}(1^\lambda, \rho)$ and $(t', \rho') \leftarrow \text{Split}(\text{pk})$ then $\rho' = \rho$; and (2) For any λ , distributions $\{(\text{sk}, \text{pk}) \mid \rho \leftarrow \mathcal{PK}_\rho, (\text{sk}, \text{pk}) \leftarrow \text{Keygen2}(1^\lambda, \rho)\}$ and $\{(\text{sk}, \text{pk}) \mid (\text{sk}, \text{pk}) \leftarrow \text{Keygen}(1^\lambda)\}$ are identical.

Security requirements on splittable KEM. Protocol **NoIC** [ABJ25] relies on mild and standard KEM properties of ciphertext security, anonymity, public key uniformity, and correctness. Specifically, it requires one-wayness under plaintext-checking attacks (OW-PCA), ciphertext anonymity under plaintext-checking attacks (ANO-PCA), and indistinguishability of public keys from elements uniformly sampled from the public key

¹³Indeed, encrypting ρ exposes the protocol to timing attacks: If the attacker eavesdrops on the communication and measures the time either party takes in the variable-time seed expansion, then it can unmask the seed with candidate passwords, expand each candidate seed, and compare the execution time to the one observed. Passwords that result in significant deviations from honest parties timings can be excluded.

¹⁴Security of **Tempo** would hold even if these two distributions are computationally indistinguishable, but in the case of ML-KEM these distributions are identical.

space (UNI-PK). Recall that OW-PCA and ANO-PCA are weakenings of respectively indistinguishability and anonymity under chosen ciphertext attacks, IND-CCA and ANO-CCA, which hold for ML-KEM under the decisional module LWE assumption, see [BDK⁺18] and [GMP22, Xag22, MX23].

In Figure 1 we define security experiments OW-PCA⁺, ANO-PCA⁺, UNI-PK⁺ which modify the above standard notions by allowing the adversary to pick the ρ part of the challenge public keys, which are then generated by $\text{Keygen2}(1^\lambda, \rho)$ instead of the standard key generation algorithm $\text{Keygen}(1^\lambda)$. This leads to the following versions of the standard KEM security notions, which all concern KEM security under *adversarially chosen seed*:

Exp OW-PCA ⁺ [KEM, $\mathcal{A}$](λ)	Exp ANO-PCA ⁺ [KEM, $\mathcal{A}$](λ)
$(\rho, \text{st}) \leftarrow \mathcal{A}(1^\lambda)$	$(\rho, \text{st}) \leftarrow \mathcal{A}(1^\lambda)$
$(\text{sk}, \text{pk}) \leftarrow \text{KEM.Keygen2}(1^\lambda, \rho)$	$(\text{sk}_0, \text{pk}_0) \leftarrow \text{KEM.Keygen2}(1^\lambda, \rho)$
$(c^*, _) \leftarrow \text{KEM.Encap}(\text{pk})$	$(\text{sk}_1, \text{pk}_1) \leftarrow \text{KEM.Keygen2}(1^\lambda, \rho)$
$K \leftarrow \mathcal{A}^{\text{PCO}_\perp(\text{sk}, \cdot, \cdot)}(\text{st}, \text{pk}, c^*)$	$b \leftarrow \{0, 1\}$
return $K == \text{KEM.Decap}(\text{sk}, c^*)$	$(c^*, _) \leftarrow \text{KEM.Encap}(\text{pk}_b)$
	$b' \leftarrow \mathcal{A}^{\text{PCO}_{c^*}(\text{sk}_0, \cdot, \cdot)}(\text{st}, \text{pk}_0, \text{pk}_1, c^*)$
Exp UNI-PK ⁺ [KEM, $\mathcal{A}$](λ)	return $b == b'$
$(\rho, \text{st}) \leftarrow \mathcal{A}(1^\lambda)$	
$(_, \text{pk}_0) \leftarrow \text{KEM.Keygen2}(1^\lambda, \rho)$	Oracle $\text{PCO}_{c^*}(\text{sk}, c, K)$
$t \leftarrow \mathcal{PK}_t$	$[plaintext\text{-}checking\text{-}oracle]$
$\text{pk}_1 \leftarrow \text{KEM.Split}^{-1}(t, \rho)$	if $c == c^*$ return $\perp$
$b \leftarrow \{0, 1\}$	return $K == \text{KEM.Decap}(\text{sk}, c)$
$b' \leftarrow \mathcal{A}(\text{st}, \text{pk}_b)$	
return $b == b'$	

Figure 1: Security experiments for a *splittable* KEM, including (1) one-wayness under plaintext-checking attacks and adversarially chosen seed (OW-PCA⁺); (2) anonymity under plaintext-checking attacks and adversarially chosen seed (ANO-PCA⁺); (3) partial pseudo-uniformity of public-keys under adversarially-chosen seed (UNI-PK⁺).

Definition 2. (Splittable KEM security for adversarially chosen seeds) We call Splittable KEM scheme KEM OW-PCA⁺-secure (‘one-way under plaintext checking attack’), ANO-PCA⁺-secure (‘anonymous under plaintext checking attack’), and UNI-PK⁺-secure (‘has pseudo-uniform public keys’), if for any efficient algorithm $\mathcal{A}$ its advantage defined respectively as:

$$\text{Adv}_{\text{KEM}, \mathcal{A}}^{\text{OW-PCA}^+}(\lambda) \stackrel{\text{def}}{=} \Pr[\text{OW-PCA}^+[\text{KEM}, \mathcal{A}](\lambda) = 1] \quad (1)$$

$$\text{Adv}_{\text{KEM}, \mathcal{A}}^{\text{ANO-PCA}^+}(\lambda) \stackrel{\text{def}}{=} 2 \cdot \Pr[\text{ANO-PCA}^+[\text{KEM}, \mathcal{A}](\lambda) = 1] - 1 \quad (2)$$

$$\text{Adv}_{\text{KEM}, \mathcal{A}}^{\text{UNI-PK}^+}(\lambda) \stackrel{\text{def}}{=} 2 \cdot \Pr[\text{UNI-PK}^+[\text{KEM}, \mathcal{A}](\lambda) = 1] - 1 \quad (3)$$

is a negligible function of λ , for security experiments specified in Fig. 1. We call scheme KEM *secure for adversarially chosen seeds* if it satisfies all three properties above.

Splittable ML-KEM. The NIST standard ML-KEM [SAB⁺22] is a secure splittable KEM under a variant of the Module Learning with Errors (MLWE) assumption underlying ML-KEM. ML-KEM relies on polynomial ring $\mathcal{R}_q^{256} = \mathcal{Z}_q[X]/(X^{256} + 1)$ for prime $q = 3329$ and noise distributions $\mathcal{B}_\eta$ over polynomials in $\mathcal{R}_q^{256}$, where coefficients are sampled from a

binomial distribution over a short range $[-\eta, \dots, \eta]$. As described earlier in the paper, the MLWE matrix is compressed using a map $\text{SampleNTT}^{\text{rep}} : \{0, 1\}^{256} \rightarrow (\mathcal{R}_q^{256})^{k \times k}$, where $\text{SampleNTT}^{\text{rep}}$ sets entry $A[i, j]$ as $\text{SampleNTT}(\rho|i|j)$ using the rejection sampling algorithm SampleNTT , see Section 2. Matrix dimension k depends on the security parameter λ , e.g. $k = 2$ for $\lambda = 128$. ML-KEM key generation $\text{Keygen}(1^\lambda)$ samples $\rho \leftarrow \{0, 1\}^{256}$ and $s, \epsilon \leftarrow \mathcal{B}_\eta^k$, sets $A = \text{SampleNTT}^{\text{rep}}(\rho)$ and $t = A \cdot s + \epsilon$, and outputs $\text{pk} = (t, \rho)$ and $\text{sk} = s$. (This view of sk is a simplification, see below, but it suffices for our explanations.)

In the *splittable* ML-KEM, algorithm Split simply divides a public key into components t in $\mathcal{PK}_t = (\mathcal{R}_q^{256})^k$ and ρ in $\mathcal{PK}_\rho = \{0, 1\}^{256}$, and algorithm $\text{Keygen2}(1^\lambda, \rho)$ performs all the steps of $\text{Keygen}(1^\lambda)$ after the choice of ρ , i.e. it picks s, ϵ , computes A and t , and outputs (sk, pk) as above. These algorithms clearly satisfy the requirements of a *Splittable KEM*, see Definition 1.

ML-KEM security under adversarially chosen seeds. The MLWE assumption underlying ML-KEM posits that these two distributions are indistinguishable:

$$\{(A, t)\}_{A \leftarrow (\mathcal{R}_q^{256})^{k \times k}, t \leftarrow (\mathcal{R}_q^{256})^k} \approx \{(A, t)\}_{A \leftarrow (\mathcal{R}_q^{256})^{k \times k}, s, \epsilon \leftarrow \mathcal{B}_\eta^k, t \leftarrow A \cdot s + \epsilon}$$

We consider an MLWE variant where the adversary interacts with a random oracle G and then outputs ρ s.t. matrix $A \in (\mathcal{R}_q^{256})^{k \times k}$ is set as $A = G(\rho)$. Note that modelling the map from ρ to A as a random oracle is the standard approach to analysing ML-KEM, and that this generalization of MLWE follows naturally this approach.¹⁵

Definition 3 (Oracle-MLWE). Assume $G : \{0, 1\}^{256} \rightarrow (\mathcal{R}_q^{256})^{k \times k}$ is a random oracle. We say that *Oracle-MLWE* is hard if for all efficient algorithms $\mathcal{A}$ it holds that $\text{Adv}_{\mathcal{A}}^{\text{O-MLWE}}(\lambda) = |\Pr[1 \leftarrow \text{Exp}_{\mathcal{A}}^0(\lambda)] - \Pr[1 \leftarrow \text{Exp}_{\mathcal{A}}^1(\lambda)]|$ is a negligible function of λ , where $\text{Exp}_{\mathcal{A}}^b(\lambda)$ runs the following algorithm:

- (1) set $(\rho, \text{st}) \leftarrow \mathcal{A}^G(1^\lambda)$,
- (2) if $b = 0$ set $t \leftarrow (\mathcal{R}_q^{256})^k$ else set $t \leftarrow A \cdot s + \epsilon$ for $A = G(\rho)$ and $s, \epsilon \leftarrow \mathcal{B}_\eta^k$,
- (3) set $b' \leftarrow \mathcal{A}^G(\text{st}, t)$ and output b' .

Theorem 1 (Splittable ML-KEM security under Oracle-MLWE). *Under the Oracle-MLWE assumption, the splittable ML-KEM described above is secure for adversarially chosen seeds, see Def. 2, in the random oracle model.*

Proof. First, note that splittable ML-KEM satisfy the same $(1 - \delta)$ -correctness and γ -spread properties as ML-KEM. The γ -spread property is defined [FO13, HHK17] for all $\text{pk} = (t, \rho)$, hence if it holds for ML-KEM then it holds also if ρ is adversarially chosen. The argument for $(1 - \delta)$ -correctness of ML-KEM does not depend on matrix A , which cancels out in the computation for the decryption error factor, see [BDK⁺18, Theorem 1]. Moreover all components of this error term are sampled from distributions independent of A , which implies correctness for adversarial ρ .

Recall that ML-KEM is constructed from a CPA-secure PKE scheme K-PKE [SAB⁺22, Section 5] which is then compiled into a CCA-secure KEM using a variant of the Fujisaki-Okamoto transform FO [FO13, HHK17], i.e. $\text{ML-KEM} = \text{FO}[\text{K-PKE}]$ [SAB⁺22, Section 6]. The public key of ML-KEM is the public key of K-PKE, consequently algorithms Split , Keygen2 apply to K-PKE as well. (Technically $\text{sk} = s$ in K-PKE, while in ML-KEM $\text{sk}' = (s, \text{pk}, h, z)$ where $h = H(\text{pk})$, and z is a PRF key used in Decap in case FO re-encryption check fails.)

Define IND-CPA^+ as KEM indistinguishability under chosen plaintext attack, but for seed chosen by adversary $\mathcal{A}$, i.e. the game sets $(\rho, \text{st}) \leftarrow \mathcal{A}(1^\lambda)$, $(\text{sk}, \text{pk}) \leftarrow \text{KEM.Keygen2}(1^\lambda, \rho)$, $(c, K_1) \leftarrow \text{KEM.Encap}(\text{pk})$, $K_0 \leftarrow \{0, 1\}^\lambda$, $b \leftarrow \{0, 1\}$, $b' \leftarrow \mathcal{A}(\text{st}, \text{pk}, c, K_b)$, and IND-CPA^+

¹⁵Indeed, $\text{SampleNTT}^{\text{rep}}$ can be shown to be indifferentiable from a random oracle that works exactly as G if the underlying XOF is itself idealized as an extendable output random oracle [ABJS24, Appendix C].

holds if $\text{Adv}_{\mathcal{A}, \text{KEM}}^{\text{IND-CPA}^+}(\lambda) = |2 \cdot \Pr[b' == b] - 1|$ is at most negligible for all efficient algorithms $\mathcal{A}$. Likewise, define ANO-CPA^+ as ANO-PCA^+ in Fig. 1 but remove $\mathcal{A}$'s access to the $\text{PCO}_\perp$ oracle.

It is straightforward to argue that K-PKE is IND-CPA^+ and ANO-CPA^+ under the Oracle-MLWE assumption. The proof of both is very similar to the proof that K-PKE is IND-CPA and ANO-CPA under MLWE [BDK⁺18], except that MLWE samples are constructed based on matrix $A = G(\rho)$ for ρ selected by the adversary. Recall that K-PKE ciphertext is $c = (u, v)$ where $u = A^T \cdot r + \epsilon_1$ and $v = t^T \cdot r + \epsilon_2 + (q/2) \cdot m$, for $m \in \{0, 1\}^{256}$ and $(\epsilon_1, \epsilon_2) \leftarrow \mathcal{B}_\eta^k \cdot \mathcal{B}_\eta$. (Values u, v are compressed by losing lower-order bits, but that is irrelevant to the present explanations.) To argue IND-CPA^+ , the first hybrid shows indistinguishability of the real t component of the public key, $t = A \cdot s + \epsilon$, from a random $t \leftarrow (\mathcal{R}_q^{256})^k$, which for $A = G(\rho)$ follows directly from the Oracle-MLWE assumption. Note that encryption can be restated as forming ciphertext $c = (u|v)$ as $c = (A')^T \cdot r + \epsilon' + m'$ for $A' = (A|v)$, $\epsilon' = (\epsilon_1|\epsilon) \leftarrow \mathcal{B}_\eta^{k+1}$ and $m' = (0^{256 \cdot k}|m)$. In the second hybrid we define an extended hash function $G' : \{0, 1\}^{256} \rightarrow (\mathcal{R}_q^{256})^{k \times k + 1}$ and for each ρ the reduction to Oracle-MLWE sets $G'(\rho)$ as $A' = [A|v]$ for random A, v . This reduction shows that under Oracle-MLWE value $\bar{c} = (A')^T \cdot r + \epsilon'$ is indistinguishable from $\bar{c} \leftarrow (\mathcal{R}_q^{256})^{k+1}$, and consequently so is the ciphertext $c = \bar{c} + m'$. This implies IND-CPA^+ and ANO-CPA^+ of K-PKE: The first holds because indistinguishability of real and random ciphertext implies lack of correlation of the ciphertext with the encrypted message m . The second holds because the same reduction can be performed for both pk_0 and pk_1 , showing that if ciphertext c is indistinguishable from random then it is uncorrelated to the public key.

The argument that splittable ML-KEM is UNI-PK^+ under the Oracle-MLWE assumption is already contained above: If $A = G(\rho)$ for adversarially chosen ρ then $t \leftarrow \text{Keygen2}(1^\lambda, \rho)$ is indistinguishable from a uniform sample $t \leftarrow \mathcal{PK}_t$.

It remains to argue OW-PCA^+ and ANO-PCA^+ . Define IND-CCA^+ as an ‘adversarial seed’ variant of IND-CCA . Recall that IND-CCA of ML-KEM is argued in [HHK17] by splitting the FO transform [FO13] into two sub-steps, T_1 and $U^\perp$, s.t. (1) if PKE_0 is $(1 - \delta)$ -correct, γ -spread, and IND-CPA then $\text{PKE}_1 = T_1[\text{PKE}_0]$ is OW-PCA , and (2) if PKE_1 is OW-PCA then $\text{KEM} = U^\perp[\text{PKE}_1]$ is IND-CCA . Both transformations also preserve $(1 - \delta)$ -correctness. As a corollary, if K-PKE is IND-CPA then $\text{ML-KEM} = (U^\perp \circ T_1)[\text{K-PKE}]$ is IND-CCA . (ML-KEM [SAB⁺22] uses a variant of $U^\perp$ which adds pk to an RO hash input, but that does not affect arguments in [HHK17].) Both arguments pertain without change to the ‘adversarial seed’ variants of these notions, i.e. same argument as (1) shows that if PKE_0 is IND-CPA^+ , $(1 - \delta)$ -correct and γ -spread, then $T_1[\text{PKE}_0]$ is OW-PCA^+ , and (2) shows that if PKE_1 is OW-PCA^+ then $U^\perp[\text{PKE}_1]$ is IND-CCA^+ . (The only change in either argument is in a final reduction to resp. IND-CPA and OW-PCA , but these work without changes if a challenge public key contains an adversarial ρ .) Since K-PKE is IND-CPA^+ , γ -spread, and $(1 - \delta)$ -correct under Oracle-MLWE this implies that ML-KEM is IND-CCA^+ , hence also OW-PCA^+ . The arguments for ANO-CCA of ML-KEM in [MX23] follow the same modular pattern, moving from PKE with ANO-PCA , $(1 - \delta)$ -correctness, and γ -spread, to $\text{KEM} = (U^\perp \circ T_1)[\text{PKE}]$ which is ANO-CCA , and this modular proof also generalizes to an argument that if PKE is ANO-PCA^+ then $\text{KEM} = (U^\perp \circ T_1)[\text{PKE}]$ is ANO-CCA^+ . $\square$

How hard is Oracle-MLWE? It is easy to see that Oracle-MLWE is asymptotically hard if MLWE is hard: The reduction selects one of $\mathcal{A}$'s RO queries in which to embed the MLWE instance, and if it guesses right then Oracle-MLWE distinguisher translates to an MLWE distinguisher. However, in exact security terms this argument incurs q_G security loss where q_G is the bound on the number of $\mathcal{A}$'s oracle queries, so to get λ -bit security based on Oracle-MLWE the ML-KEM parameters would have to increase to assure $(\lambda + \log_2(q_G))$ -bit security of MLWE. On the other hand, this is only a lower-bound

on Oracle-MLWE hardness. Note that oracle G samples matrix A from a space of size $|\mathcal{R}_q^{k \times k}| = 3329^{256 \cdot k^2} \geq 3329^{1024} \sim 2^{11981}$. Therefore, even if $q_G = 2^{80}$, the adversary who chooses ρ s.t. $A = G(\rho)$ can select A out of at most 2^{80} random samples in the space of size at least 2^{11981} , so it could effectively influence < 0.0067 fraction of the bits of A (while the rest would still be uniform), and we do not know of any attack on MLWE which obtains advantage from such a skew in the distribution of A .

3.2 Protocol Tempo

We describe **Tempo** protocol in Figure 2. Protocol **Tempo** follows protocol **NoIC** [ABJ25], except for how Alice’s public key is password-encrypted. We show the password-encryption block schematically in Figure 3. The main point is that part ρ of the ML-KEM public key $pk = (t, \rho)$ is sent in the clear and only part t is password-encrypted. Therefore, in particular, the timing information gathered from Alice’s or Bob’s operation $A = \text{SampleNTT}^{\text{rep}}(\rho)$, which is variable-time in the FIPS 203 standard [NIS23], leaks no password-related information. However, the password is still present in **Tempo** in the inputs that determine the keys on the 2-round Feistel, hence round functions H_1, H_2 **must be implemented in constant time**. H_2 hashes onto bitstrings so constant-time implementation can be done using standard hash functions. H_1 hashes onto group $\mathcal{PK}_t = (\mathcal{R}_q^{256})^k$, and it can be implemented using any **constant-time SampleNTT algorithm**, e.g. algorithm **FLS**, **RMD**, or **SMD** of Section 2. This way **Tempo** can instantiate KEM with an off-the-shelf ML-KEM implementation, which does not need any modification, and all the PAKE-specific password-dependent operations will be implemented to satisfy the standard constant-time discipline.

Tempo is a generalization of **NoIC**: If $\mathcal{PK}$ is a group and $\text{Split}(pk)$ outputs $(pk, \perp)$, i.e. it does not actually decompose the public key, then we recover the **NoIC** construction.¹⁶ However, since our goal is to compile ML-KEM into a PAKE protocol without changing the variable-time rejection sampling procedure specified in the standard [NIS23], we focus on the **Tempo** protocol instantiated with ML-KEM and a splitting function that decomposes the public key into the pair (t, ρ) , denoted $\text{Tempo}[\text{ML-KEM}, \text{Split}(pk) \rightarrow (t, \rho)]$. The following theorem captures the security of protocol **Tempo**:

Theorem 2. *Assume KEM is a splittable KEM scheme (see Def. 1) and $\text{Split} : \mathcal{PK} \rightarrow \mathcal{PK}_t \times \mathcal{PK}_\rho$ is an associated splitting function. If KEM is $(1 - \delta)$ -correct, OW-PCA⁺, ANO-PCA⁺, and UNI-PK⁺ (see Def. 2) then $\text{Tempo}[\text{KEM}, \text{Split}]$, shown in Fig. 2, UC-realizes the PAKE functionality $\mathcal{F}_{\text{PAKE}}$ of Canetti et al. [CHK⁺05].*

Proof (sketch). To show that **Tempo** UC-realizes $\mathcal{F}_{\text{PAKE}}$ (for completeness we include the standard UC PAKE functionality $\mathcal{F}_{\text{PAKE}}$ of [CHK⁺05] in appendix), we (1) show a simulator **SIM** that interacts with $\mathcal{F}_{\text{PAKE}}$ and presents to the environment $\mathcal{Z}$ a view that is consistent with that of the ‘real world’, where honest parties execute protocol **Tempo**; and (2) prove that $\mathcal{Z}$ cannot distinguish between the ‘real world’ and the ‘ideal world’ orchestrated by this **SIM**.

Simulator **SIM** is shown in Figures 4 and 5. Simulator **SIM** is a variant of the simulator used in the proof of **NoIC** [ABJ25], and we highlight the differences between the two simulators in the figures. Below we provide a quick overview of the simulator and the differences from **NoIC**, and we point out how they affect the security proof of [ABJ25].

When $\mathcal{F}_{\text{PAKE}}$ reports that Alice initiated the protocol, the simulator doesn’t know Alice’s password, but it chooses $\text{apk}_{\text{fsid}} = (u, v, \rho)$ at random, see step 3 in Fig. 4. Since ρ is independent of the password and (u, v) is a 2-Feistel password-encryption of a pseudorandom ring element t , this matches the real-world view.

¹⁶One can also render **NoIC** instantiation OQUAKE proposed in [VJW25] as $\text{Tempo}[\text{ML-KEM-768}, \text{Split}(pk) \rightarrow (\text{KemelonNR}(pk), \perp)]$, in which case $\mathcal{PK}_t = \{0, 1\}^m$ for some m and operation $\odot$ is an XOR.

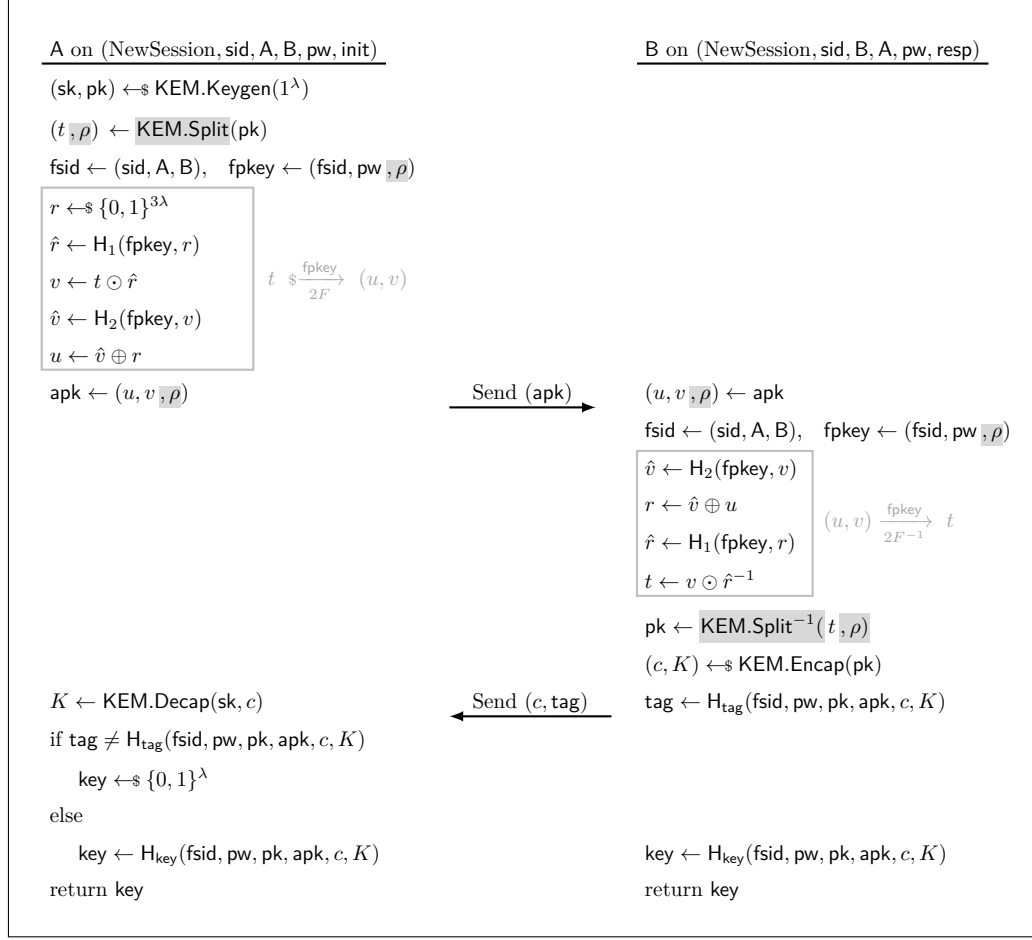


Figure 2: Protocol Tempo, assuming a KEM with splittable keys, see Def. 1, whose public keys can be split into two components, an element t of group $\mathcal{PK}_t$ and a fixed-sized bitstring ρ in $\mathcal{PK}_\rho = \{0, 1\}^{\text{poly}(\lambda)}$. Only t is password-encrypted using a two-round Feistel network, while ρ is transmitted in the clear. For comparison with PAKE protocol NoIC [ABJ25], the above figure defines Tempo if one includes dark gray shaded text and it defines NoIC if one omits these text fragments. Group operations are represented by $\odot$, the bitstring XOR operation by $\oplus$, and the inverse operation by $(\cdot)^{-1}$. $H_1, H_2, H_{tag}, H_{key}$ are hash functions with respective ranges $\mathcal{PK}_t, \{0, 1\}^{3\lambda}, \{0, 1\}^{2\lambda}$, and $\{0, 1\}^\lambda$.

When a message is sent to Bob in the real world, Bob always encapsulates using the public key recovered using 2-Feistel password-decryption from the received apk . The simulator must recover the same public key that Bob would obtain in the real world, potentially one related to Alice’s public key if the adversary exploits the malleability of the 2-round Feistel by reusing r and correctly guessing Bob’s password—this is described as a *related key attack* in [ABJ25], see step 6a in Fig. 5. In Tempo, we chose to include ρ as input to H_1 and H_2 oracles, thereby ensuring that related key attacks can only be mounted with respect to the same ρ output by honest Alice. By the committing property of 2-Feistel observed in [MRR20], ciphertext (u, v) can be output of fresh adversarial password-encryption on just one password, which is extractable by the simulator, see Steps 5 and 6 in Fig. 5. If neither of these active attacks is present, or if they fail because functionality $\mathcal{F}_{PAKE}$ returns that the adversary’s effective password guess pw^* is

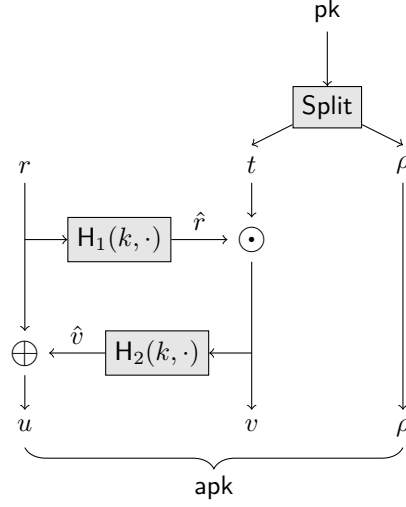


Figure 3: A visual representation of how the Tempo protocol password-encrypts the public key pk . The algorithm **Split** decomposes pk into a group element $t \in \mathcal{PK}_t$ and a fixed-length bitstring $\rho \in \mathcal{PK}_\rho$. The hash functions H_1 and H_2 take $k = \text{fsid} \parallel \text{pw} \parallel \rho$ as part of the input, ensuring domain separation and binding of ρ transmitted in clear. $\oplus$ is the XOR operation over bitstrings and $\odot$ is the group operation in $\mathcal{PK}_t$.

incorrect, see Step 8a in Fig. 5, then the real-world Bob would use **KEM.Encap** on key $pk[\text{pw}_B] = (t[\text{pw}_B], \rho)$ for $t[\text{pw}_B] = 2F_{\text{pw}_B}^{-1}(u, v)$, where each key $pk[\text{pw}_B]$ could be generated via **KEM.Keygen2**($1^\lambda, \rho$). Since the simulator doesn't know pw_B , it uses instead a key generated on the fly via **KEM.Keygen2**($1^\lambda, \rho$), see Step 7 in Fig. 5. The simulated and real executions are indistinguishable by **ANO-PCA**⁺, i.e. **KEM** anonymity under adversarially chosen seeds, and Bob's output key is secure by **OW-PCA**⁺, i.e. **KEM** one-wayness under adversarial seeds.

When Alice receives response (c, tag) the simulator acts depending on whether the session was passive, or actively attacked, see Steps 9-11. In the latter case the authentication tag enables password extraction, and the session key is set via the $\mathcal{F}_{\text{PAKE}}$ functionality, but this process is exactly as in the **NoIC** simulator [ABJ25].

When servicing hash queries **SIM** checks for abort conditions which would make password-extraction from active attacks ambiguous, see Steps 2a-2b in Fig. 4. Moreover, **SIM** embeds public keys with knowledge of the corresponding secret keys in H_1/H_2 queries that implement password-decryption $2F^{-1}(\cdot)$ of Alice's apk_{fsid} . Specifically, whenever the adversary queries H_2 on fsid, ρ, v matching Alice's apk_{fsid} , **SIM** programs a public key into the H_1 table. This process is virtually the same as in the **NoIC** simulator [ABJ25], except that each sampled key pair must pertain to the same ρ specified in apk_{fsid} , hence **SIM** generates these keys via **KEM.Keygen2**($1^\lambda, \rho$), see Step 2c in Fig. 4.

Finally, the proof that $\mathcal{F}_{\text{PAKE}}$ and **SIM** present to $\mathcal{Z}$ an indistinguishable view from the real interaction follows the corresponding proof of **NoIC**, except in the game hops that pertain to active attacks on Bob because it is only there that we have to deal with the adversarially chosen seed ρ .¹⁷ Each of the affected arguments deals with Step 7 in Fig. 5, i.e. Bob's response to a failed or non-existent active attack. Technically this is split in the proof into two parts. First, games G_{10} - G_{12} show that the simulation is correct in case Bob receives $\text{apk} = \text{apk}_{\text{fsid}}$. Here ρ is not chosen by the attacker but the simulator generates Bob's possible keys via **Keygen2**, hence the argument involves reductions to

¹⁷Indeed note that, on Alice's side, we are always guaranteed to operate on a public key that was honestly sampled.

Initialization: Initialize empty tables $\text{TH}_1, \text{TH}_2, \text{TH}_{\text{tag}}, \text{TH}_{\text{key}}$, and initialize $\text{apk}_{\text{fsid}}, \text{transcriptB}_{\text{fsid}}$ as $\perp$ and $\text{forwS}_{\text{fsid}}, \text{backS}_{\text{fsid}}$ as empty sets for all fsid .

On $\mathcal{A}$'s query x to H_{ind} for $\text{ind} \in \{1, 2, \text{tag}, \text{key}\}$:

If $\text{TH}_{\text{ind}}[x] \neq \perp$ return $\text{TH}_{\text{ind}}[x]$ (and terminate), else sample $\text{TH}_{\text{ind}}[x] \leftarrow \$ \text{Space}[\text{H}_{\text{ind}}]$ and return this value after first performing the following steps:

1. If $\mathcal{A}$ queries H_{tag} and $\exists x' \neq x$ s.t. $\text{TH}_{\text{tag}}[x] = \text{TH}_{\text{tag}}[x']$ then *abort* [A1].
2. If $\mathcal{A}$ queries H_2 then parse $x = (\text{fsid}, \text{pw}, \rho, v)$ and do the following:
 - (a) For all v^*, pw^* s.t. there is a record $\text{TH}_2[\text{fsid}, \text{pw}^*, \rho, v^*] \neq \perp$, verify that $\text{TH}_2[\text{fsid}, \text{pw}, \rho, v] \oplus \text{TH}_2[\text{fsid}, \text{pw}, \rho, v^*] \neq \text{TH}_2[\text{fsid}, \text{pw}^*, \rho, v] \oplus \text{TH}_2[\text{fsid}, \text{pw}^*, \rho, v^*]$, else *abort* [A2a].
// abort if query results in two passwords explaining same apk in the related key attack (see step 6a)
 - (b) For all r s.t. $\hat{r} = \text{TH}_1[\text{fsid}, \text{pw}, \rho, r] \neq \perp$, set $u = \text{TH}_2[\text{fsid}, \text{pw}, \rho, v] \oplus r$,
 - verify that $(u, v, \rho) \neq \text{apk}_{\text{fsid}}$, else *abort* [A2b1],
 - verify that $(\cdot, \cdot, (u, v)) \notin \text{forwS}_{\text{fsid}}$, else *abort* [A2b2],
 set $\text{pk} = (v \odot \hat{r}^{-1}, \rho)$, set $\text{apk} = (u, v, \rho)$ and add $(\text{pw}, \text{pk}, \text{apk})$ to $\text{forwS}_{\text{fsid}}$.
// record pk encrypted in apk under pw for any u implied by current TH1 records; abort if apk = apk_{fsid} or apk is already associated in set forwS_{fsid} with some other pw, pk pair.
 - (c) If $\exists u$ s.t. $\text{apk}_{\text{fsid}} = (u, v, \rho)$ set $r \leftarrow u \oplus \text{TH}_2[\text{fsid}, \text{pw}, \rho, v]$ (note that $\text{TH}_1[\text{fsid}, \text{pw}, \rho, r] = \perp$ by abort condition [A2b1]),
 pick $(\text{sk}, \text{pk}) \leftarrow \$ \text{KEM.Keygen2}(1^\lambda, \rho)$, set $(t, \rho) \leftarrow \text{KEM.Split}(\text{pk})$,
 $\text{TH}_1[\text{fsid}, \text{pw}, \rho, r] \leftarrow t^{-1} \odot v$ and add $(\text{pw}, (\text{sk}, \text{pk}))$ to set $\text{backS}_{\text{fsid}}$.
// program KEM key in backward H2 query matching apk_{fsid};

On (NewSession, sid, A, B, init) from $\mathcal{F}_{\text{PAKE}}$:

3. Set $\text{fsid} \leftarrow (\text{sid}, \text{A}, \text{B})$, sample $(u, v, \rho) \leftarrow \$ \{0, 1\}^{3\lambda} \times \mathcal{PK}_t \times \mathcal{PK}_\rho$, verify that $\forall \text{pw}$ $\text{TH}_2[\text{fsid}, \text{pw}, \rho, v] = \perp$ (else *abort* [A3]), record $(\text{fsid}, \text{init})$, set $\text{apk} \leftarrow (u, v, \rho)$, set $\text{apk}_{\text{fsid}} \leftarrow \text{apk}$, and send apk to $\mathcal{A}$.

On (NewSession, sid, B, A, resp) from $\mathcal{F}_{\text{PAKE}}$:

4. Set $\text{fsid} \leftarrow (\text{sid}, \text{A}, \text{B})$ and record $(\text{fsid}, \text{resp})$.

Figure 4: Simulator for the security proof of Tempo protocol (part 1). The highlighted portions indicate significant differences with respect to the NoIC simulator [ABJ25].

OW-PCA⁺ and ANO-PCA⁺. Second, games G_{18} and G_{19} at the end of the proof show that the simulation is correct in case Bob receives an adversarial $\text{apk} \neq \text{apk}_{\text{fsid}}$, but an active attack this message represents fails, i.e. a password guess message apk represents is incorrect. Here the adversary chooses ρ in apk , so the arguments involve reductions to OW-PCA⁺, ANO-PCA⁺, and also UNI-PK⁺ because of programming Keygen2-generated keys into $2F^{-1}(\cdot)$ outputs. (We refer the reader to [ABJ25] for the proof details, and to the appendix for an overview of the game hops and their differences from [ABJ25].) $\square$

On $\mathcal{A}$'s message $\text{apk} = (u, v, \rho)$ to party $\mathcal{B}$'s session sid :

If $\exists$ record $(\text{fsid}, \text{resp})$ for $\text{fsid} = (\text{sid}, \mathcal{A}, \mathcal{B})$ then delete it and do the following:

5. If $\exists \text{pw}^*, \text{pk}^*$ s.t. $(\text{pw}^*, \text{pk}^*, \text{apk}) \in \text{forwS}_{\text{fsid}}$, go to step 8. // *chosen key attack*
6. Parse $((u, v, \rho), (\bar{u}, \bar{v}, \bar{\rho})) \leftarrow (\text{apk}, \text{apk}_{\text{fsid}})$ and if $\rho = \bar{\rho} \wedge \text{apk} \neq \text{apk}_{\text{fsid}}$:
 - (a) if $\exists \text{pw}^*$ s.t. $\text{TH}_2[\text{fsid}, \text{pw}^*, \rho, v] \oplus \text{TH}_2[\text{fsid}, \text{pw}^*, \bar{\rho}, \bar{v}] = u \oplus \bar{u}$,
 set $\bar{t} \leftarrow \bar{v} \odot (\text{TH}_1[\text{fsid}, \text{pw}^*, \bar{\rho}, \text{TH}_2[\text{fsid}, \text{pw}^*, \bar{\rho}, \bar{v}] \oplus \bar{u}])^{-1}$,
 set $t^* \leftarrow \bar{t} \odot v \odot (\bar{v})^{-1}$, $\text{pk}^* \leftarrow \text{KEM.Split}^{-1}(t^*, \bar{\rho})$, then go to step 8.
 // *related key attack*
 - (b) else send $(\text{TestPwd}, \text{sid}, \mathcal{B}, \perp)$ to $\mathcal{F}_{\text{PAKE}}$. // *unmindful attack*
7. Pick $(\text{sk}, \text{pk}) \leftarrow \text{KEM.Keygen2}(1^\lambda, \rho)$, set $(c, K) \leftarrow \text{KEM.Encap}(\text{pk})$, and $\text{tag} \leftarrow \{0, 1\}^{2\lambda}$. Set $\text{transcriptB}_{\text{fsid}} \leftarrow (\text{apk}, (c, \text{tag}))$, send (c, tag) to $\mathcal{A}$ and $(\text{NewKey}, \text{sid}, \mathcal{B}, \perp)$ to $\mathcal{F}_{\text{PAKE}}$, return control to $\mathcal{Z}$.
 // *no attack, failed attack (step 8a), or unmindful attack (step 6b)*
8. Send $(\text{TestPwd}, \text{sid}, \mathcal{B}, \text{pw}^*)$ to $\mathcal{F}_{\text{PAKE}}$, receive $(\text{TestPwd}, \text{sid}, \text{resp})$ and:
 - (a) if $\text{resp} = \text{wrong}$ go to step 7. // *failed attack*
 - (b) if $\text{resp} = \text{correct}$ set $(c, K) \leftarrow \text{KEM.Encap}(\text{pk}^*)$, $\text{tag} \leftarrow \text{H}_{\text{tag}}(\text{fsid}, \text{pw}^*, \text{pk}^*, \text{apk}, c, K)$, $\text{key}^* \leftarrow \text{H}_{\text{key}}(\text{fsid}, \text{pw}^*, \text{pk}^*, \text{apk}, c, K)$ (query $\text{H}_{\text{tag}}, \text{H}_{\text{key}}$ as $\mathcal{A}$), send (c, tag) to $\mathcal{A}$ and $(\text{NewKey}, \text{sid}, \mathcal{B}, \text{key}^*)$ to $\mathcal{F}_{\text{PAKE}}$, and set $\text{transcriptB}_{\text{fsid}} \leftarrow (\text{apk}, (c, \text{tag}))$. // *successful attack*

On $\mathcal{A}$'s message (c, tag) to party $\mathcal{A}$'s session sid :

If $\exists$ record $(\text{fsid}, \text{init})$ for $\text{fsid} = (\text{sid}, \mathcal{A}, \mathcal{B})$ then delete it, set $\text{apk} \leftarrow \text{apk}_{\text{fsid}}$, and do:

9. If $(\text{apk}, (c, \text{tag})) = \text{transcriptB}_{\text{fsid}}$ then send $(\text{NewKey}, \text{sid}, \mathcal{A}, \perp)$ to $\mathcal{F}_{\text{PAKE}}$. // *passive adversary*
10. If $(\text{apk}, (c, \text{tag})) \neq \text{transcriptB}_{\text{fsid}}$ and $\exists \text{pw}^*, \text{pk}, K$ s.t. $\text{tag} = \text{TH}_{\text{tag}}[\text{fsid}, \text{pw}^*, \text{pk}, \text{apk}, c, K]$ and $K = \text{KEM.Decap}(\text{sk}, c)$ for sk s.t. $(\text{pw}^*, (\text{sk}, \text{pk})) \in \text{backS}_{\text{fsid}}$, then set $\text{key}^* \leftarrow \text{H}_{\text{key}}(\text{fsid}, \text{pw}^*, \text{pk}, \text{apk}, c, K)$ (query H_{key} as $\mathcal{A}$), and send $(\text{TestPwd}, \text{sid}, \mathcal{A}, \text{pw}^*)$ and $(\text{NewKey}, \text{sid}, \mathcal{A}, \text{key}^*)$ to $\mathcal{F}_{\text{PAKE}}$.
 // *active attack*
11. Else send $(\text{TestPwd}, \text{sid}, \mathcal{A}, \perp), (\text{NewKey}, \text{sid}, \mathcal{A}, \perp)$ to $\mathcal{F}_{\text{PAKE}}$. // *failed attack*

Figure 5: Simulator for the security proof of Tempo protocol (part 2). The highlighted portions indicate significant differences with respect to the NoIC simulator [ABJ25].

Performance comparison with NoIC. To recap, this paper offers two approaches for building a PAKE from ML-KEM while mitigating timing attacks: (1) We ensure that ML-KEM public-key expansion is implemented in constant-time (either by implementing rejection sampling itself in constant time or using alternative sampling methods that deviate from FIPS 203 [NIS23], as discussed in Section 2), and we additionally repurpose the constant-time rejection sampler to implement H_1 in password-encryption; or (2) we propose the Tempo protocol to make the seed ρ public and use one of the constant-time

sampling methods discussed solely to instantiate H_1 .

This raises the question of how these two approaches compare in efficiency. To answer it, we benchmark both solutions by implementing them in the ARM Cortex-M4 family of microcontrollers, using the `pqm4` benchmarking framework [KPR⁺] and a Nucleo-L4R5ZI¹⁸ development board. The Nucleo-L4R5ZI board is the default board selected by `pqm4` to produce comparable measurements for post-quantum KEMs and signature schemes. It is resource constrained, so large cryptographic libraries such as OpenSSL are known to be a challenge in this setting [KRSS19]. Accordingly, we focus on the standalone versions of algorithms `FLS` and `SMD`. In any case, and as discussed in Section 2, algorithm `RMD` that we left out from these benchmarks offers worse performance than algorithm `FLS`, as shown in Table 1, nor does it have the benefit of being functionally equivalent to `SampleNTT` as specified in the standard [NIS23].

The results appear in Table 2, and we also report ML-KEM benchmarks as a baseline for reference. For intuition on how to read the reported measurements, recall that key generation, encapsulation and decapsulation all expand ρ into matrix $A \in (\mathbb{Z}_q^{256})^{k \times k}$. (Decapsulation does so as part of the re-encryption step in the Fujisaki-Okamoto transform.) Also note that the t -part of the public key lies in $(\mathbb{Z}_q^{256})^k$, so masking it via H_1 is expected to be about k times faster than expanding ρ into A using the same sampling method.

The source code for these `NoIC` and `Tempo` implementations is open-sourced (available at <https://github.com/afonsoarriaga/ProjectTempo>) and so they can be easily reproduced.

4 Conclusions

A great deal of attention has been devoted in recent years to building PAKEs from KEMs. These works follow the Encrypted Key Exchange paradigm by encrypting the public key under a password. However, timing attacks have largely been ignored so far, despite their well known practical relevance [VR20]. This is visible, for example, in the `CAKE/OCAKE` [BCP⁺23] proposal to instantiate the ideal cipher over an arbitrary finite domain using the cycle walking method [BR02], and in the `CHIC` [ABJS24] proposal to repurpose ML-KEM rejection sampling procedure `SampleNTT` to instantiate the random oracle that produces the public-key mask. Rejection sampling and cycle walking methods do not translate naturally to constant-time code, a fact overlooked in prior work and critical for security. As such, countermeasures against timing attacks, and their performance impact, were also poorly understood. The issue is further compounded when the KEM itself is not constant-time (key generation, encapsulation and decapsulation algorithms), as is the case for essentially all ML-KEM implementations we are aware of, with respect to the public key expansion step, due to their reliance on `SampleNTT`.

This paper first investigates how to make ML-KEM constant-time so that it can be used to instantiate KEM-to-PAKE compilers such as `CAKE/OCAKE`, `CHIC`, and `NoIC` [ABJ25]. We propose a constant-time implementation of `SampleNTT` that preserves the functionality of the standard specification [NIS23], as well as two alternatives that deviate from the standard. Following the approach suggested in `CHIC`, these samplers can also be repurposed to instantiate the random oracle used to derive the public-key mask in `NoIC` and `CHIC`. We also note that the `OQUAKE` approach, proposed in an IETF Internet-Draft [VJW25], instantiates `NoIC` with `BUKEM`, which augments ML-KEM with a uniform, fixed-length encoding of public keys. This is meant to simplify the password-encryption part of the construction, but it introduces computational and bandwidth costs, and it does not eliminate timing side channels. We give benchmarking results that permit comparing the various alternatives.

¹⁸<https://www.st.com/en/evaluation-tools/nucleo-l4r5zi.html>

Table 2: Benchmarks (in cycles on a Cortex M4 Nucleo-L4R5ZI board, averaged over 100 runs $\pm$ standard deviations) comparing **NoIC** and **Tempo** instantiated with ML-KEM. We restrict our attention to implementations that are standalone and time-safe. Results for ML-KEM reference code are included for baseline reference.

Algorithm FLS			
<i>Fixed Loop Sampler</i> (constant-time refactor of [NIS23, Algorithm 7])			
Avg cycles to execute NoIC PAKE			
Algorithm FLS used both to (re-)generate matrix A and instantiate H_1			
	start	response	end
NoIC[ML-KEM-512]	21 864 462 $\pm$ 17	22 323 521 $\pm$ 19	15 290 858 $\pm$ 11
NoIC[ML-KEM-768]	43 473 335 $\pm$ 19	44 099 762 $\pm$ 20	33 534 883 $\pm$ 2
NoIC[ML-KEM-1024]	72 215 747 $\pm$ 17	73 019 488 $\pm$ 18	58 912 526 $\pm$ 20
Avg cycles to execute Tempo PAKE			
Algorithm FLS used only to instantiate H_1			
ML-KEM reference implementation without modifications			
	start	response	end
Tempo[ML-KEM-512]	7 811 197 $\pm$ 1896	8 270 248 $\pm$ 1902	1 247 949 $\pm$ 1898
Tempo[ML-KEM-768]	11 854 590 $\pm$ 1928	12 480 987 $\pm$ 1922	1 936 396 $\pm$ 1924
Tempo[ML-KEM-1024]	16 023 652 $\pm$ 4663	16 827 413 $\pm$ 4668	2 740 706 $\pm$ 4671
Algorithm SMD			
<i>Single Modular Division</i> (new constant-time method)			
Avg cycles to execute NoIC PAKE			
Algorithm SMD used both to (re-)generate matrix A and instantiate H_1			
	start	response	end
NoIC[ML-KEM-512]	3 247 444 $\pm$ 16	3 706 500 $\pm$ 17	2 879 509 $\pm$ 15
NoIC[ML-KEM-768]	6 239 288 $\pm$ 19	6 865 688 $\pm$ 20	5 609 324 $\pm$ 19
NoIC[ML-KEM-1024]	10 159 019 $\pm$ 19	10 962 759 $\pm$ 18	9 267 136 $\pm$ 19
Avg cycles to execute Tempo PAKE			
Algorithm SMD used only to instantiate H_1			
ML-KEM reference implementation without modifications			
	start	response	end
Tempo[ML-KEM-512]	1 605 498 $\pm$ 1917	2 064 550 $\pm$ 1915	1 247 926 $\pm$ 1915
Tempo[ML-KEM-768]	2 546 709 $\pm$ 3220	3 173 110 $\pm$ 3226	1 937 037 $\pm$ 3220
Tempo[ML-KEM-1024]	3 612 553 $\pm$ 4620	4 416 318 $\pm$ 4622	2 740 962 $\pm$ 4621
Avg cycles to execute ML-KEM reference implementation			
	keygen	encaps	decaps
ML-KEM-512	596 338 $\pm$ 2209	701 146 $\pm$ 2210	889 195 $\pm$ 2210
ML-KEM-768	987 580 $\pm$ 3244	1 137 080 $\pm$ 3244	1 386 842 $\pm$ 3245
ML-KEM-1024	1 536 967 $\pm$ 2874	1 708 715 $\pm$ 2878	2 020 960 $\pm$ 2879

CAKE/OCAKE and NoIC can be instantiated with any KEM satisfying the required security properties, enabling crypto-agility. Nevertheless, ML-KEM is an inevitable candidate in practice due to its efficiency, compact public keys, and standardization. Therefore, we also propose **Tempo**, a protocol similar to **NoIC** but that deliberately sends the seed ρ that expands to matrix **A** in the clear, making the variable-time sampling computation public again. This approach requires assuming that ML-KEM retains security, anonymity, and public-key uniformity properties even under adversarially chosen seeds. We relate these properties to a new hardness assumption, Oracle-MLWE, which is asymptotically equivalent to the decision-MLWE problem, although the reduction loses tightness by a factor proportional to the number of oracle queries made by the adversary. In practice,

since the random oracle is instantiated via an eXtensible Output Function (that a real-world attacker can compute offline), assuming only the hardness of the decision-MLWE problem requires accounting for this loss with a large query bound. We are not aware, however, of any attack that exploits searching for a seed ρ that expands to a matrix A that weakens the MLWE problem. Nevertheless, two possible mitigations to counter the tightness loss are: (1) instantiate **Tempo** with a higher ML-KEM security level than required by the target application to compensate for the security loss; or (2) fix a global seed per application domain, akin to a common reference string assumption (without any trapdoor knowledge required). In any case, the performance gains of **Tempo** over **NoIC[ML-KEM]** justify considering this design choice, especially for resource-constrained environments.

Finally, we benchmark these protocol variants and provide a baseline for future PAKE implementations. The fastest option is **Tempo** when H_1 is instantiated using the modular-reduction method in Algorithm **SMD**, with a cost of about twice that of ML-KEM without timing protection.

Acknowledgments

We would like to thank Roman Langrehr for helpful discussions on the Oracle-MLWE assumption, Tiago Oliveira for sharing his insights on benchmarking cryptographic algorithms, and Michael Rosenberg for contributing a pull request to the project’s repository that improved the implementation of algorithm RMD. We also thank the anonymous reviewers of CHES for their constructive feedback, which contributed to improving this manuscript.

Afonso Arriaga received support from the Luxembourg National Research Fund (FNR) under the CORE project (C21/IS/16221219/ImPAKT).

References

- [AAA⁺24] Nouri Alnahawi, Jacob Alperin-Sheriff, Daniel Apon, Gareth T. Davies, and Alexander Wiesmaier. NICE-PAKE: On the security of KEM-based PAKE constructions without ideal ciphers. *Cryptology ePrint Archive*, Report 2024/1957, 2024.
- [ABJ25] Afonso Arriaga, Manuel Barbosa, and Stanislaw Jarecki. NoIC: PAKE from KEM without ideal ciphers. *Cryptology ePrint Archive*, Report 2025/231, 2025.
- [ABJS24] Afonso Arriaga, Manuel Barbosa, Stanislaw Jarecki, and Marjan Skrobot. C’est Très CHIC: A compact password-authenticated key exchange from lattice-based KEM. In Kai-Min Chung and Yu Sasaki, editors, *ASIACRYPT 2024, Part V*, volume 15488 of *LNCS*, pages 3–33. Springer, Singapore, December 2024.
- [AHH21] Michel Abdalla, Björn Haase, and Julia Hesse. Security analysis of CPace. In Mehdi Tibouchi and Huaxiong Wang, editors, *ASIACRYPT 2021, Part IV*, volume 13093 of *LNCS*, pages 711–741. Springer, Cham, December 2021.
- [AHR24] Nouri Alnahawi, Kathrin Hövelmanns, Andreas Hülsing, and Silvia Ritsch. Towards post-quantum secure PAKE - A tight security proof for OCAKE in the BPR model. In Markulf Kohlweiss, Roberto Di Pietro, and Alastair R. Beresford, editors, *CANS 2024, Part II*, volume 14906 of *LNCS*, pages 191–212. Springer, Singapore, September 2024.

- [AMO⁺26] Afonso Arriaga, David Mestel, Jan Oupický, Peter Browne Rønne, and Marjan Škrobot. HIC is all you need: Practical post-quantum password-authenticated public key encryption. Cryptology ePrint Archive, Paper 2026/020, *to appear at CT-RSA'26*, 2026.
- [AP05] Michel Abdalla and David Pointcheval. Simple password-based encrypted key exchange protocols. In Alfred Menezes, editor, *CT-RSA 2005*, volume 3376 of *LNCS*, pages 191–208. Springer, Berlin, Heidelberg, February 2005.
- [BCP03] Emmanuel Bresson, Olivier Chevassut, and David Pointcheval. Security proofs for an efficient password-based key exchange. In Sushil Jajodia, Vijayalakshmi Atluri, and Trent Jaeger, editors, *ACM CCS 2003*, pages 241–250. ACM Press, October 2003.
- [BCP⁺23] Hugo Beguinet, Céline Chevalier, David Pointcheval, Thomas Ricosset, and Mélissa Rossi. GeT a CAKE: Generic transformations from key encapsulation mechanisms to password authenticated key exchanges. In Mehdi Tibouchi and Xiaofeng Wang, editors, *ACNS 2023, Part II*, volume 13906 of *LNCS*, pages 516–538. Springer, Cham, June 2023.
- [BDK⁺18] Joppe W. Bos, Léo Ducas, Eike Kiltz, Tancrede Lepoint, Vadim Lyubashevsky, John M. Schanck, Peter Schwabe, Gregor Seiler, and Damien Stehlé. CRYSTALS - kyber: A CCA-secure module-lattice-based KEM. In *2018 IEEE European Symposium on Security and Privacy*, pages 353–367. IEEE Computer Society Press, April 2018.
- [BM92] Steven M. Bellovin and Michael Merritt. Encrypted key exchange: Password-based protocols secure against dictionary attacks. In *1992 IEEE Symposium on Security and Privacy*, pages 72–84. IEEE Computer Society Press, May 1992.
- [BMP00] Victor Boyko, Philip D. MacKenzie, and Sarvar Patel. Provably secure password-authenticated key exchange using Diffie-Hellman. In Bart Preneel, editor, *EUROCRYPT 2000*, volume 1807 of *LNCS*, pages 156–171. Springer, Berlin, Heidelberg, May 2000.
- [BPR00] Mihir Bellare, David Pointcheval, and Phillip Rogaway. Authenticated key exchange secure against dictionary attacks. In Bart Preneel, editor, *EUROCRYPT 2000*, volume 1807 of *LNCS*, pages 139–155. Springer, Berlin, Heidelberg, May 2000.
- [BR02] John Black and Phillip Rogaway. Ciphers with arbitrary finite domains. In Bart Preneel, editor, *CT-RSA 2002*, volume 2271 of *LNCS*, pages 114–130. Springer, Berlin, Heidelberg, February 2002.
- [CDMS10] Jean-Sébastien Coron, Yevgeniy Dodis, Avradip Mandal, and Yannick Seurin. A domain extender for the ideal cipher. In Daniele Micciancio, editor, *TCC 2010*, volume 5978 of *LNCS*, pages 273–289. Springer, Berlin, Heidelberg, February 2010.
- [CFHL21] Kai-Min Chung, Serge Fehr, Yu-Hsuan Huang, and Tai-Ning Liao. On the compressed-oracle technique, and post-quantum security of proofs of sequential work. In Anne Canteaut and François-Xavier Standaert, editors, *EUROCRYPT 2021, Part II*, volume 12697 of *LNCS*, pages 598–629. Springer, Cham, October 2021.

- [CHK⁺05] Ran Canetti, Shai Halevi, Jonathan Katz, Yehuda Lindell, and Philip D. MacKenzie. Universally composable password-based key exchange. In Ronald Cramer, editor, *EUROCRYPT 2005*, volume 3494 of *LNCS*, pages 404–421. Springer, Berlin, Heidelberg, May 2005.
- [DS16] Yuanxi Dai and John P. Steinberger. Indifferentiability of 8-round Feistel networks. In Matthew Robshaw and Jonathan Katz, editors, *CRYPTO 2016, Part I*, volume 9814 of *LNCS*, pages 95–120. Springer, Berlin, Heidelberg, August 2016.
- [FO13] Eiichiro Fujisaki and Tatsuaki Okamoto. Secure integration of asymmetric and symmetric encryption schemes. *Journal of Cryptology*, 26(1):80–101, January 2013.
- [GMP22] Paul Grubbs, Varun Maram, and Kenneth G. Paterson. Anonymous, robust post-quantum public key encryption. In Orr Dunkelman and Stefan Dziembowski, editors, *EUROCRYPT 2022, Part III*, volume 13277 of *LNCS*, pages 402–432. Springer, Cham, May / June 2022.
- [GSV24a] Felix Günther, Douglas Stebila, and Shannon Veitch. Obfuscated key exchange. In Bo Luo, Xiaojing Liao, Jun Xu, Engin Kirda, and David Lie, editors, *ACM CCS 2024*, pages 2385–2399. ACM Press, October 2024.
- [GSV24b] Felix Günther, Douglas Stebila, and Shannon Veitch. Kameleon Encodings. Internet-Draft draft-veitch-kameleon-00, Internet Engineering Task Force, November 2024. Work in Progress.
- [HHK17] Dennis Hofheinz, Kathrin Hövelmanns, and Eike Kiltz. A modular analysis of the Fujisaki-Okamoto transformation. In Yael Kalai and Leonid Reyzin, editors, *TCC 2017, Part I*, volume 10677 of *LNCS*, pages 341–371. Springer, Cham, November 2017.
- [HHKR25] Kathrin Hövelmanns, Andreas Hülsing, Mikhail Kudinov, and Silvia Ritsch. CAKE requires programming - on the provable post-quantum security of (O)CAKE. Cryptology ePrint Archive, Report 2025/458, 2025.
- [HL19] Björn Haase and Benoît Labrique. AuCPace: Efficient verifier-based PAKE protocol tailored for the IIoT. *IACR TCHES*, 2019(2):1–48, 2019.
- [HR24] Julia Hesse and Michael Rosenberg. PAKE combiners and efficient post-quantum instantiations. *IACR Cryptol. ePrint Arch.*, page 1621, 2024.
- [HvO22] Feng Hao and Paul C. van Oorschot. SoK: Password-authenticated key exchange - theory, practice, standardization and real-world lessons. In Yuji Suga, Kouichi Sakurai, Xuhua Ding, and Kazue Sako, editors, *ASIACCS 22*, pages 697–711. ACM Press, May / June 2022.
- [JRX25] Jake Januzelli, Lawrence Roy, and Jiayu Xu. Under what conditions is encrypted key exchange actually secure? In Serge Fehr and Pierre-Alain Fouque, editors, *EUROCRYPT 2025, Part II*, volume 15602 of *LNCS*, pages 451–481. Springer, Cham, May 2025.
- [KPR⁺] Matthias J. Kannwischer, Richard Petri, Joost Rijneveld, Peter Schwabe, and Ko Stoffelen. PQM4: Post-quantum crypto library for the ARM Cortex-M4. <https://github.com/mupq/pqm4>.

- [KRSS19] Matthias J. Kannwischer, Joost Rijneveld, Peter Schwabe, and Ko Stoffelen. pqm4: Testing and benchmarking NIST PQC on ARM Cortex-M4. Cryptology ePrint Archive, Report 2019/844, 2019.
- [LBES25] Patrick Longa, Joppe W. Bos, Stephan Ehlen, and Douglas Stebila. FrodoKEM: key encapsulation from learning with errors. Internet-Draft draft-longa-cfrg-frodokem-00, Internet Engineering Task Force, March 2025. Work in Progress.
- [LL24] You Lyu and Shengli Liu. Hybrid password authentication key exchange in the UC framework. *IACR Cryptol. ePrint Arch.*, page 1630, 2024.
- [MRR20] Ian McQuoid, Mike Rosulek, and Lawrence Roy. Minimal symmetric PAKE and 1-out-of-N OT from programmable-once public functions. In Jay Ligatti, Xinming Ou, Jonathan Katz, and Giovanni Vigna, editors, *ACM CCS 2020*, pages 425–442. ACM Press, November 2020.
- [MvOV96] Alfred J. Menezes, Paul C. van Oorschot, and Scott A. Vanstone. *Handbook of Applied Cryptography*. CRC Press, Boca Raton, Florida, 1996.
- [MX23] Varun Maram and Keita Xagawa. Post-quantum anonymity of Kyber. In Alexandra Boldyreva and Vladimir Kolesnikov, editors, *PKC 2023, Part I*, volume 13940 of *LNCS*, pages 3–35. Springer, Cham, May 2023.
- [NIS23] NIST. FIPS203, Module-Lattice-based Key-Encapsulation Mechanism Standard. Federal Information Processing Standards Publication, 2023.
- [PZ23] Jiaxin Pan and Runzhi Zeng. A generic construction of tightly secure password-based authenticated key exchange. In Jian Guo and Ron Steinfeld, editors, *ASIACRYPT 2023, Part VIII*, volume 14445 of *LNCS*, pages 143–175. Springer, Singapore, December 2023.
- [SAB⁺22] Peter Schwabe, Roberto Avanzi, Joppe Bos, Léo Ducas, Eike Kiltz, Tancrede Lepoint, Vadim Lyubashevsky, John M. Schanck, Gregor Seiler, Damien Stehlé, and Jintai Ding. CRYSTALS-KYBER. Technical report, National Institute of Standards and Technology, 2022. available at <https://csrc.nist.gov/Projects/post-quantum-cryptography/selected-algorithms-2022>.
- [SGJ23] Bruno Freitas Dos Santos, Yanqi Gu, and Stanislaw Jarecki. Randomized half-ideal cipher on groups with applications to UC (a)PAKE. In Carmit Hazay and Martijn Stam, editors, *EUROCRYPT 2023, Part V*, volume 14008 of *LNCS*, pages 128–156. Springer, Cham, April 2023.
- [Tib14] Mehdi Tibouchi. Elligator squared: Uniform points on elliptic curves of prime order as uniform random strings. In Nicolas Christin and Reihaneh Safavi-Naini, editors, *FC 2014*, volume 8437 of *LNCS*, pages 139–156. Springer, Berlin, Heidelberg, March 2014.
- [Unr23] Dominique Unruh. Towards compressed permutation oracles. In Jian Guo and Ron Steinfeld, editors, *ASIACRYPT 2023, Part IV*, volume 14441 of *LNCS*, pages 369–400. Springer, Singapore, December 2023.
- [VJW25] Jelle Vos, Stanislaw Jarecki, and Christopher A. Wood. Hybrid Post-Quantum Password Authenticated Key Exchange. Internet-Draft draft-vos-cfrg-pqpake-00, Internet Engineering Task Force, April 2025. Work in Progress.
- [VR20] Mathy Vanhoef and Eyal Ronen. Dragonblood: Analyzing the dragonfly handshake of WPA3 and EAP-pwd. In *2020 IEEE Symposium on Security and Privacy*, pages 517–533. IEEE Computer Society Press, May 2020.

- [Xag22] Keita Xagawa. Anonymity of NIST PQC round 3 KEMs. In Orr Dunkelman and Stefan Dziembowski, editors, *EUROCRYPT 2022, Part III*, volume 13277 of *LNCS*, pages 551–581. Springer, Cham, May / June 2022.
- [Zha19] Mark Zhandry. How to record quantum queries, and applications to quantum indistinguishability. In Alexandra Boldyreva and Daniele Micciancio, editors, *CRYPTO 2019, Part II*, volume 11693 of *LNCS*, pages 239–268. Springer, Cham, August 2019.

Appendix

A SampleNTT as specified in FIPS 203

Algorithm 7 (SampleNTT: Variable-time rejection sampling)

This algorithm is as specified in FIPS 203 [SAB⁺22, Algorithm 7]. The highlighted parts indicate an optional bound on the while-loop, as per [SAB⁺22, Appendix B]. Including this bound preserves the uniform distribution of the output coefficients but introduces a statistical failure probability of 2^{-161} .

Input: byte array $B \in \mathbb{B}^{34}$

Output: array $a \in \mathbb{Z}_q^{256}$

```

1: ctx ← XOF.Init()
2: ctx ← XOF.Absorb(ctx, B)
3: j ← 0
4: iter ← 0
5: while j < 256 and iter < 280 do
6:   (ctx, C) ← XOF.Squeeze(ctx, 3)           ▷ generating a 3-byte sample C
7:   d1 ← C[0] + 256 · (C[1] mod 16)
8:   d2 ← ⌊C[1]/16⌋ + 16 · C[2]
9:   if d1 < q then
10:    a[j] ← d1
11:    j ← j + 1
12:   end if
13:   if d2 < q and j < 256 then
14:    a[j] ← d2
15:    j ← j + 1
16:   end if
17:   iter ← iter + 1
18: end while
19: return a

```

B UC functionality for PAKE

New Session. On $(\text{NewSession}, \text{sid}, P_i, P_j, \text{pw}, \text{role})$ from party P_i: • Ignore this query if two or more records of the form $(\text{sid}, \dots)$ already exist. • Else record $(\text{sid}, P_i, P_j, \text{fresh}, \text{pw}, \perp)$, send $(\text{NewSession}, \text{sid}, P_i, P_j, \text{role})$ to $\mathcal{A}$.
Test Password Guess. On $(\text{TestPwd}, \text{sid}, P_i, \text{pw}^*)$ from adversary $\mathcal{A}$: • Retrieve record $\text{recvar} = (\text{sid}, P_i, P_j, \text{fresh}, \text{pw}, \perp)$, abort if no such record exists. • If $\text{pw}^* = \text{pw}$, then update field <i>status</i> in record recvar to compromised and send $(\text{TestPwd}, \text{sid}, \text{correct})$ to $\mathcal{A}$. • Else update field <i>status</i> in record recvar to interrupted and send $(\text{TestPwd}, \text{sid}, \text{wrong})$ to $\mathcal{A}$.
Session Key. On $(\text{NewKey}, \text{sid}, P_i, \text{key}^*)$ from adversary $\mathcal{A}$ where $\text{key}^* = \lambda$: • Retrieve record $\text{recvar} = (\text{sid}, P_i, P_j, \text{status}, \text{pw}, \perp)$, abort if no such record exists or if <i>status</i> = completed. • If <i>status</i> = compromised, set $\text{key} \leftarrow \text{key}^*$. • If <i>status</i> = fresh and $\exists$ record $(\text{sid}, P_j, P_i, \text{completed}, \text{pw}, \text{key}')$ whose <i>status</i> switched from fresh to completed when P_j received $(\text{NewKey}, \text{sid}, \text{key}')$, set $\text{key} \leftarrow \text{key}'$. • Else set $\text{key} \leftarrow \mathbb{S}\{0, 1\}^\lambda$. • Update <i>status</i> in recvar to completed and output $(\text{NewKey}, \text{sid}, \text{key})$ to P_i.

Figure B.1: The ideal UC PAKE functionality $\mathcal{F}_{\text{PAKE}}$ of Canetti et al. [CHK⁺05].

C Game hops for the security proof of Theorem 2

Below we provide a brief guideline to the game hops encountered in the proof for security of protocol *Tempo*, i.e. a proof of Theorem 2, and in Table C.1 we summarize these hops and mark the ones that differ from the ones in the proof of *NoIC* in [ABJ25].

Game G_1 (real world): This is the real world em parties execute the protocol and the simulator provides oracle access to all hash functions, storing all queries in tables.

Game G_2 (hash aborts): The game adds abort conditions [A1], [A2a], [A2b1], [A2b2]. The probability of these bad events happening is statistically negligible.

Game G_3 (kem correctness): If Alice and Bob derive different KEM keys from matching ciphertext and key pair, abort. This hop relies on KEM correctness. Since Alice picks the public key and we consider only passive attacks, the adversary has no control over the choice of ρ .

Game G_4 (key copy if pw, apk, c, tag all match): If the adversary is passive and $\text{pw}_A = \text{pw}_B$ then Alice's session key can be copied from Bob. From G_3 , it is guaranteed that both parties derive the same key.

Game G_5 (random key_A if pw, apk, c match but tag $\neq$ tag_B): The tag must be invalid since it's computed on the same inputs and we've discarded collisions in G_2 .

Game G_6 (random key_A if tag not output by H_{tag}): The probability that a tag not produced by the random oracle is valid is statistically negligible.

Game G_7 (random key_A if tag was computed on matching $\text{apk}_{\text{fsid}}, c$, but wrong pw or pk): Since G_2 , abort condition [A1] prevents collision on H_{tag} , so nothing really changes.

Game G_8 (random key_B , tag_B if pw , apk match): The ciphertext conceals key_B , which is used to compute tag_B as well. The PCA oracle lets the reduction check whether an adversarially crafted ciphertext/tag pair would be accepted by Alice. Since we consider only passive attacks on Bob (i.e., matching apk) OW-PCA suffices.

Game G_9 (independent pk_B if pw , apk match): The ciphertext is already independent of key_B . In this hop, we make it independent of pk_B as well. We are still treating passive attacks against Bob, therefore ANO-PCA suffices.

Game G_{10} (embed pk with knowledge of sk for each decryption attempt on apk_{fsid}): The simulator embeds a pk with knowledge of the corresponding sk for each decryption attempt of Alice's $\text{apk}_{\text{fsid}} = (u, v, \rho)$, i.e, by calling $H_2(\text{fsid}||\text{pw}||\rho, v)$. Since ρ is fixed, the simulator must use Keygen2 and the reduction receives a challenge for a fixed ρ .

Game G_{11} (random key_B , tag_B if apk matches): Bob selects a random key_B and tag_B upon receiving Alice's apk_{fsid} , regardless of whether the passwords match. The matching-password case was treated in G_8 , so this change only affects $\text{pw}_B \neq \text{pw}_A$. Since G_{10} , a pk is embedded in these cases, enabling a reduction to OW-PCA⁺.

Game G_{12} (independent pk_B if apk match): Bob encapsulates with an independent pk if he receives Alice's apk_{fsid} , regardless of whether their passwords match. Similarly, this is reduceable to ANO-PCA⁺.

Game G_{13} (sample apk_{fsid} at random): Alice first picks apk_{fsid} at random and then associates it with pk_A . Changing the order of operations (apk_{fsid} decryption instead of pk_A encryption) might trigger the abort events introduced in G_2 , but the probability is statistically negligible.

Game G_{14} (postpone pk embedding): Alice picks apk_{fsid} at random, postponing the embedding of pk_A . Abort conditions might not be triggered because of this change, but again this is a statistically negligible event.

Game G_{15} (random key_A if apk_{fsid} was never decrypted under pw_A): Alice selects a random key_A if the adversary never decrypted her apk_{fsid} under the correct password pw_A . Note that pk_A is an input to create tag . We argue that an authenticator tag is invalid if $\text{pk}_A = (t, \rho)$ has not been generated yet, as the probability of guessing it before its generation is tied to the min-entropy of t .

Game G_{16} (chosen key attack): The simulator checks for a chosen-key attack on Bob; if detected, it recovers the adversary's pk and lets Bob proceed using it.

Game G_{17} (related key attack): The simulator checks for a related key attack on Bob; if detected, it recovers the corresponding pk , extracting ρ from the RO records, computing t^* related to Alice's $\text{apk}_{\text{fsid}} = (\bar{t}, \rho)$, and using Split^{-1} to merge ρ and t^* ; it then lets Bob proceed using the recovered pk .

Game G_{18} (random key_B and tag_B , regardless of apk): We are now dealing with active attacks against Bob. This game hop follows the same strategy as described in NoIC proof, but since the adversary can select the seed, the reduction is against OW-PCA⁺ and UNI-PK⁺.

Game G_{19} (independent pk_B , regardless of apk): We remain in the active attack setting against Bob, but now Bob encapsulates using an independent key. As in the NoIC proof, the hop follows the same strategy. However, since the adversary chooses the seed, the reduction targets ANO-PCA⁺ and UNI-PK⁺.

Table C.1: Sequence of games and associated advantage bounds in the security proof of Tempo. Any change from the original NoIC proof is marked in the **Update** column. Here, ϵ denotes a statistically negligible probability, and 0 indicates a bridge hop with no effect on the adversary’s advantage.

Game	Description	Advantage	Update
G_1	real world	-	no
G_2	abort on hash bad events	ϵ	no
G_3	abort on correctness fail	correctness	no
G_4	Alice copies key from Bob if adversary is passive	0	no
G_5	Alice selects a random key when the transcript matches except for a mismatch in the transmitted tag value	0	no
G_6	Alice selects a random key when the transcript matches, except for a mismatch in the transmitted tag value, which was not even produced by the RO	ϵ	no
G_7	Alice selects a random key if the received tag was computed on wrong pw or wrong pk	0	no
G_8	Bob selects a random key and tag if he receives Alice’s apk and his password matches hers	OW-PCA	no
G_9	Bob encapsulates with an independent pk if he receives Alice’s apk and his password matches hers	ANO-PCA	no
G_{10}	the simulator embeds a pk with knowledge of the corresponding sk for each decryption attempt of Alice’s apk	UNI-PK ⁺	yes
G_{11}	Bob selects a random key and tag if he receives Alice’s apk , regardless of whether their passwords match	OW-PCA ⁺	yes
G_{12}	Bob encapsulates with an independent pk if he receives Alice’s apk , regardless of whether their passwords match	ANO-PCA ⁺	yes
G_{13}	Alice first picks apk at random and then associates it with pk (with knowledge of sk)	ϵ	no
G_{14}	Alice picks apk at random, postponing the embedding of pk	ϵ	no
G_{15}	Alice selects a random key if the adversary never tried to decrypt her apk under the correct password	ϵ	yes
G_{16}	the simulator checks for a chosen-key attack on Bob; if detected, it recovers the adversary’s pk and let Bob proceed using it	0	no
G_{17}	the simulator checks for a related key attack on Bob; if detected, it recovers the corresponding pk related to Alice’s, and lets Bob proceed using it	0	no
G_{18}	Bob selects a random key and tag , regardless of whether he receives Alice’s apk	OW-PCA ⁺ & UNI-PK ⁺	yes
G_{19}	Bob encapsulates with an independent pk , regardless of whether he receives Alice’s apk	ANO-PCA ⁺ & UNI-PK ⁺	yes